CSS SIMULATOR

Abstract

The aim of this document is to describe the CSS simulator and how to use it.

DATE : 17/04/2026

ISSUE : V1

Author(s)	Simone Urbano Jacques Girard Louis Lolive Lucas Hervier Pierre Bacquet
Checker(s)	Benjamin Deporte
Approver	Benjamin Deporte

Table of Contents

1 Background

CSS project (2023 → 2026) is led by the French Institute for Technological Research (IRT) Saint Exupéry in Toulouse.

CSS will include contributions from leading institutional, academic and commercial partners in the space, cybersecurity and technology sectors : CNES, LAAS-CNRS, Thales Alenia Space, Starion France, Gate Watcher, Ecole de l'Air. Starion France is responsible for the technical coordination of the project.

The objective is to mature the technological blocks necessary to improve the state-of-the-art in cybersecurity for space systems.

One of the goal is to build **a state-of-the-art Space System Simulation platform for Cybersecurity R&D**, where advanced AI techniques for attack and defence can be developed and evaluated.

In order to improve the resilience of space systems, multiple technological elements are investigated:

- Space System Simulation (Constellation of CubeSats)
- CTI for Space
- Automatic attack generation
- On Board agent for threat detection
- On Ground Probe for automatic threat detection
- On Ground AI agent (reinforcement learning) for automatic threat detection and isolation

This document describes the space system simulation element, that is currently based on NASA NOS3 simulator.

The choice of NOS3 as baseline for the development of a space system simulator representing a constellation of CubeSats is related to the following factors:

- Platform based on open source software (NASA CFS, COSMOS, NASA 42)
- Full CCSDS stack implementation (only physical RF layer is missing)
- Platform has already proven its relevance for cybersecurity research and development.
- Flight Software (NASA CFS) has already been used in flight for several NASA missions.

1.1 NASA NOS3 baseline

The simulator has been developed based on NASA NOS3 version 1.6.2. (released 07/08/2023) : https://github.com/nasa/nos3/releases/tag/v1_06_02

The reference mission used in the simulator is STF-1 mission : https://www.nasa.gov/simulation-to-flight-1/

2 NASA NOS3 Architecture

Nasa NOS3 simulator gather multiple open source projects such as

- NASA CFS (Flight Software)
- COSMOS (Mission Control System)
- NASA 42 (Flight dynamics and visualization)
- NASA Cryptolib (encryption layer)

to create a single multi-purpose, modular and highly customizable space system simulator.

Figure 1 presents the general architecture of the simulator. Figure 2 presents the NASA cFS architecture (common software bus with subscribing applications and services). Figure 3 presents the FSW/GSW/SIMS architecture used in NOS3 for the CubeSat components (Arducam example).

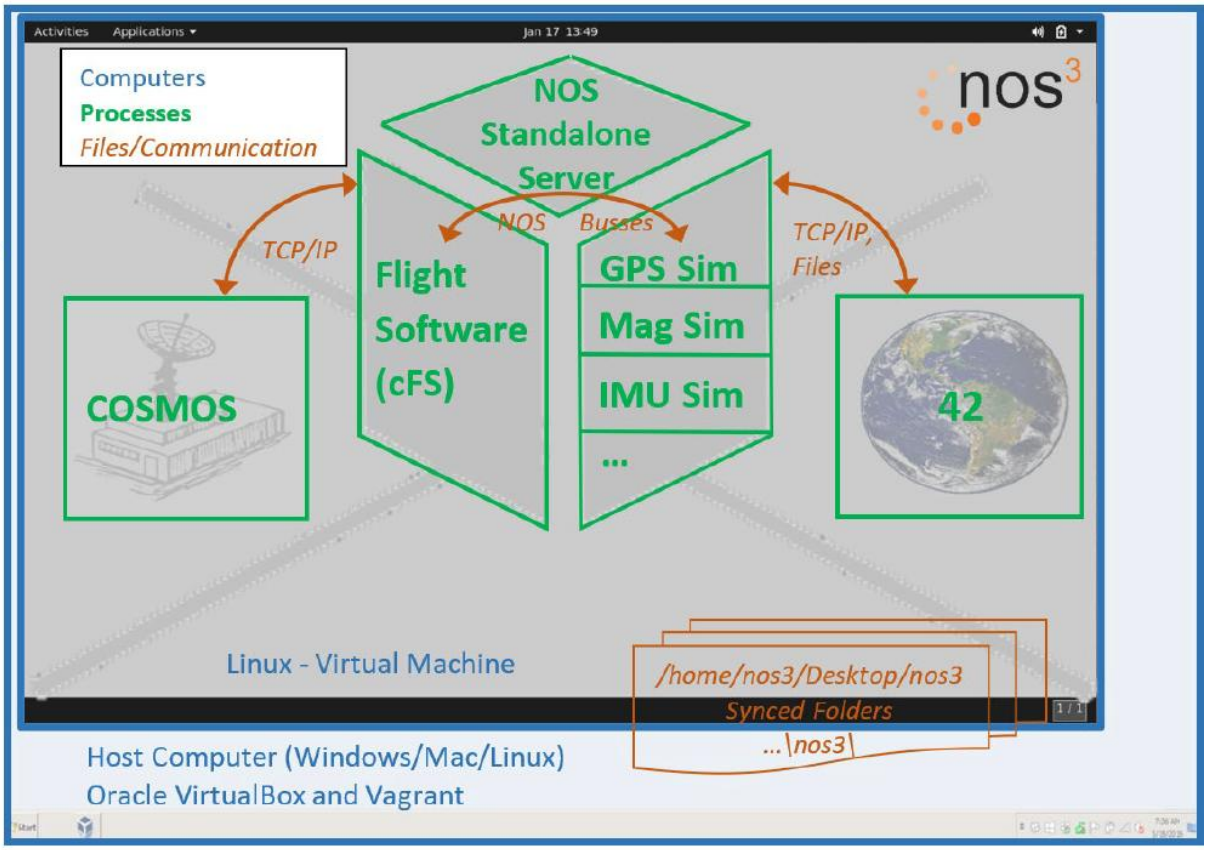


Figure 1. General architecture of NOS3 simulator.

More information about the current state of NOS3 can be found here : https://ntrs.nasa.gov/citations/20240004699.

Or on the GitHub repository of the project https://github.com/nasa/nos3.

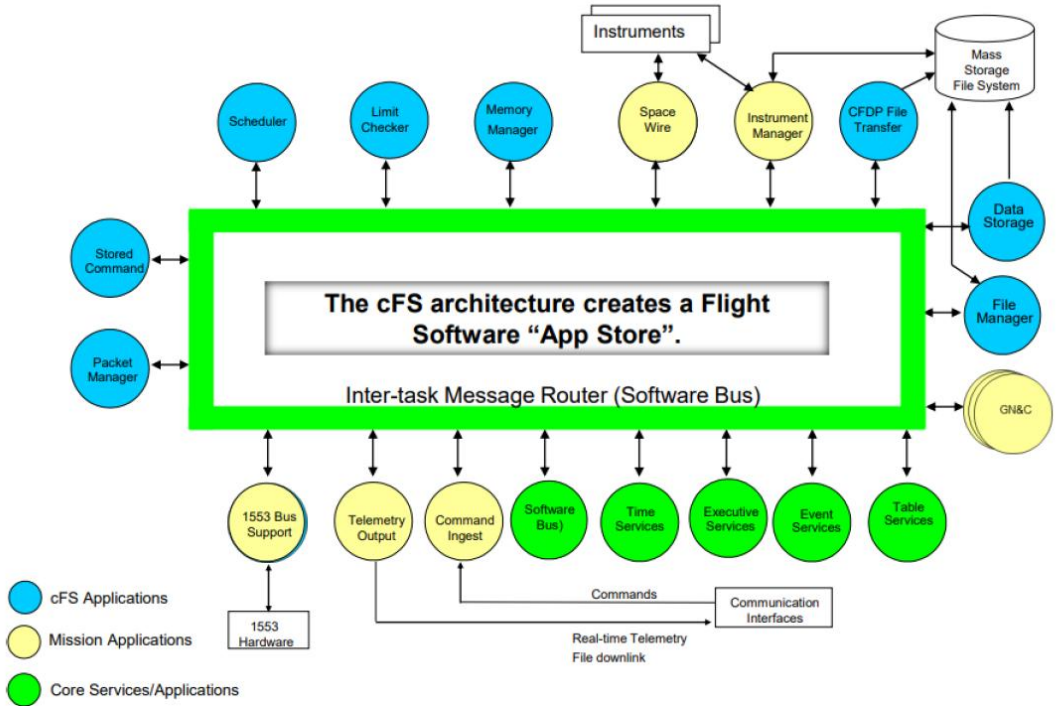


Figure 2. NASA cFS architecture.

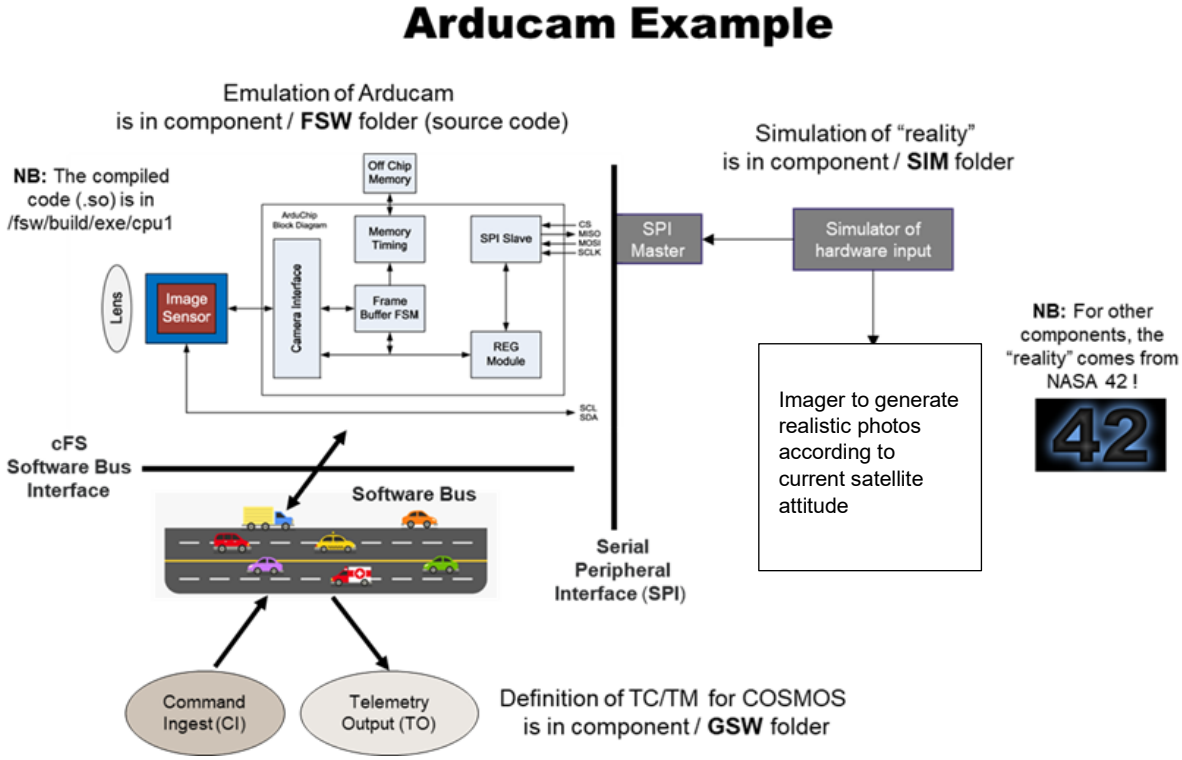


Figure 3. Example of FSW/GSW/SIM integration.

3 NASA NOS3 – Multi VMs approach

NASA NOS3 v1.6.2 is provided to be deployed on a single Virtual Machine (VM) using Vagrant (the VM is based on Ubuntu 20.04.6 LTS).

In the context of CSS project, it has been decided to split the simulator on multiple VM, in order to increase the modularity and to have the possibility to have different operators on different VMs. In particular, it has been chosen to use

- 1 VM for each satellite (NASA CFS).
- 1 VM for the Mission Control System (MCS, ground segment based on COSMOS).
- 1 VM for the flight dynamics and visualization part (NASA 42).

Figure 4 presents the chosen configuration of the simulator. The CCSDS stack used by NOS3 is transported over UDP between the MCS and the Satellites. The space environment simulation performed by NASA 42 can provide relevant information to the satellite flight software via TCP sockets.

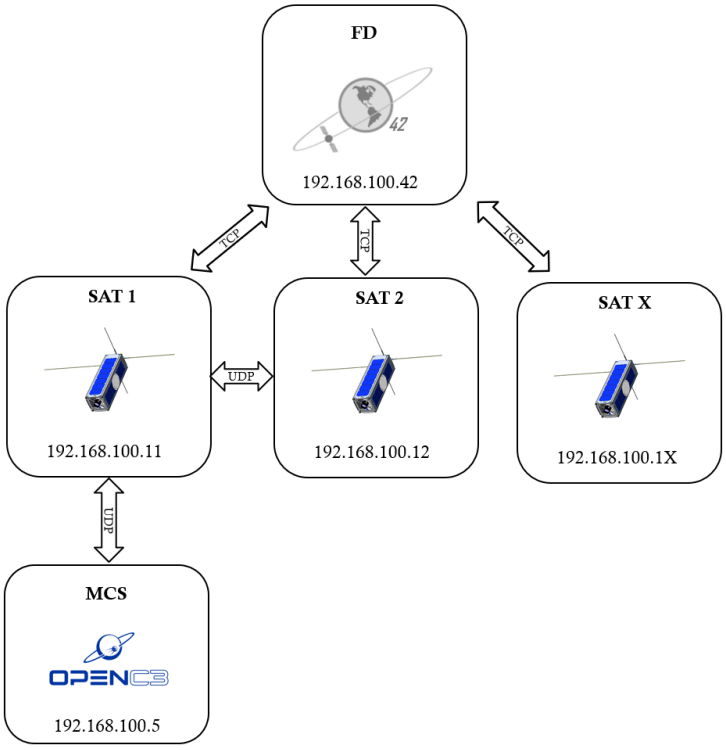


Figure 4. Multi VMs approach.

In Figure 5 one can see that the simulator can be scaled by simply adding new VMs to create more complex scenarios.

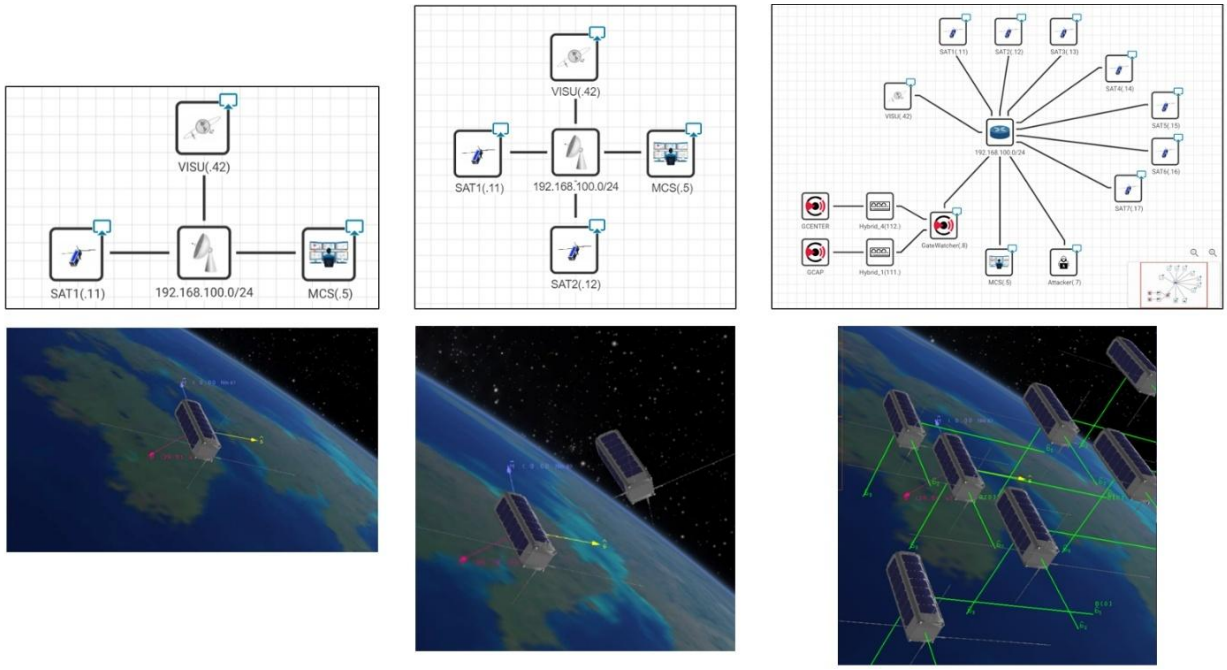


*Figure 5. From simple to more complex scenarios. Scenarios are created using **CITEF** by Nexova.*

It is important to mention here that the chosen hosting environment for the simulator development and testing is CITEF (Cyber Integration Test and Evaluation Framework) by Nexova (Nexova | Ahead of the curve), a state-of-the-art cyber range that can handle complex scenarios including several VMs. However, the CSS simulator is independent from CITEF and the user can choose the orchestrator or virtualization software he prefers to run the CSS simulator. This will be clarified in the deployment procedure section.

4 Transfer Frame Layer

NOS3 v1.6.2 implements the CCSDS stack up to SPP layer (see Figure 6). The Transfer Frame layer (TC/TM TF) was not operational and several adaptations of NOS3 code were required to activate the TF layer.

The TC/TM TF layer is a requirement of CSS project in terms of representativeness. Moreover, it delivers the source or destination spacecraft ID information, which is essential to route throughout a constellation. For this reason, several adaptations have been made in NOS3 code to reactivate the TC/TM TF layer.

NB: The physical layer (RF link) is not simulated in NOS3. CCSDS TF are passed over UDP to represent the RF link. For this reason, one can use a standard “tcpdump” command to get a capture (.pcap) of the simulator traffic.

Example on VM MCS (192.168.100.5):

```
sudo tcpdump -i eth0 -w udp_traffic.pcap udp
```

This captures all UDP packets on interface *eth0* and writes them to *udp_traffic.pcap* in binary pcap format, which can be opened for example with Wireshark.

Dissectors are also provided to the user to easy the packets analysis. In particular, in *CSS_Attacks/Dissectors* folder.

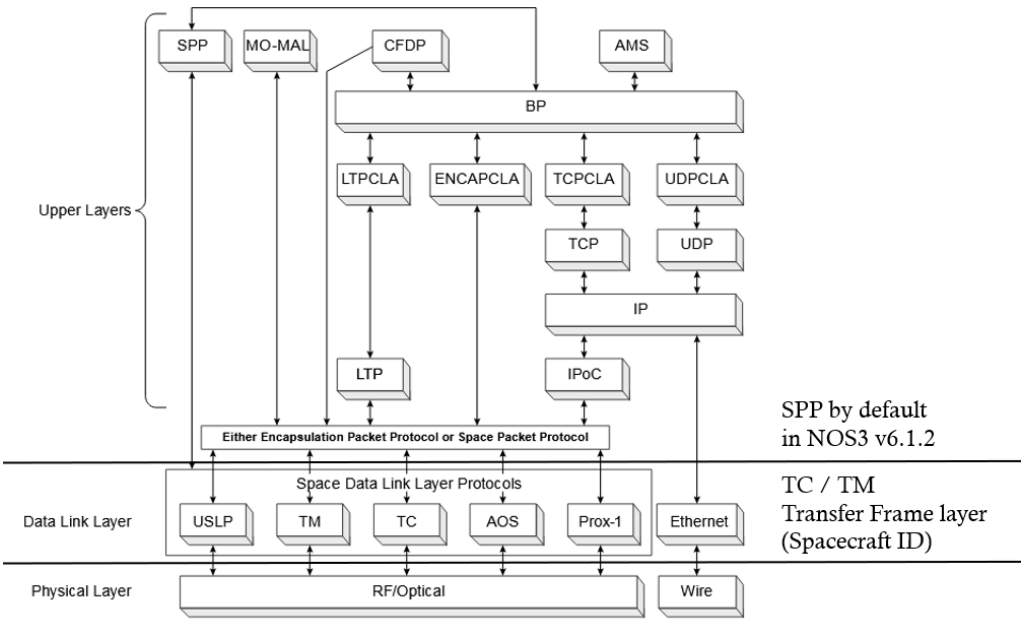


Figure 2-1: SPP Context within the CCSDS Protocol Stack

Figure 6. CCSDS stack and NOS3.

4.1 CCSDS and NOS3

It is important to clarify the specificities of CCSDS implementation in NOS3.

Indeed, for TM the code should represent CCSDS 132.0-B-3 and for TC CCSDS 232.0-B-4.

In NOS3 and CSS implementation we observe that :

- TC frames only include one TC
- The TC frame header is followed by a one-byte Segment Header, then a SP, and finally a two-bytes Frame Error Control Field.
- The NOS3 implementation offers several encryption/authentication options. We tested the following:
    - VC ID = 0. Plain mode (no encryption), unauthenticated (no MAC-type SDLS postambule). The SDLS header is reduced to 4 bytes and contains a Security Parameter Index (SPI) of 1 on the first two bytes; this SPI of 1 corresponds to a plain mode security association.
    - VC ID = 4. AES-GCM-256 encrypted and authenticated mode. 14-byte SDLS header with SPI = 4 on the first two bytes and 12 null bytes for vector initialization. Postambule MAC on the last 16 bytes.
- Frames of about 1500 bytes (including the 4-bytes Attached Synchronization Marker) are sent generally every 100 ms. With a 1500-byte MTU on Ethernet interfaces, IP fragmentation occurs (see *fsw/apps/to/fsw/examples/udp_tf/to_platform_cfg.h*).
- As specified in the CCSDS 132.0-B-3 standard, the OCF flag in the frame header, set to 0, indicates that the frame does not contain a CLCW (4 bytes) at the end. This feature of the standard implies that TC transmission occurs without flow control, which is unusual for ground missions.
- On VC Id = 1, Source Packets (SPs), in plain text, are sent sequentially without SDLS headers or SDLS postambules. One or more SPs can be contained within a frame.
- OID (Only Idle Data) frames have a constant data field (it starts with 0xff480ec0). Two violations to the norm:
    - The pseudo-random sequence should not be reset with each OID frame (see CCSDS 132.0-B-3 §4.1.4.6.2.1 and 4.1.4.6.2.2).
    - It should start with 0xffffffff6d… the first time (see CCSDS 132.0-B-3 §4.1.4.6.2.2).This is not a major issue because this pseudo-random sequence:
        - Is primarily useful for preventing the transmission of only 0s or only 1s at the RF level.
        - Even though it is constant from one frame to the next, it prevents information leakage through OID frames.
- Non-idle SPs do have a Secondary Header. The Secondary Header and the SP data are not in PUS format (PUS format is used by ESA).
- Idle SPs, mostly encountered at the end of TM frames, do not have a Secondary Header (see 133.0-B-2 §4.1.3.3.3.4) and are sent with a data area that is the same pseudo-random sequence as those of OID TM frames (The CCSDS 133.0-B-2 §1.6.1.5 standard allows complete freedom on this point).

NB: Examples of dissectors to analyse the simulator traffic are provided in *CSS_Attacks* folder.

5 General architecture

It is important to note that we have added some software elements in NOS3 in order to handle **a multi spacecraft configuration**.

In particular, the following onboard components:

- **ISL/RM** – Inter Satellite Link / Routing Machine, which provides a simple and effective routing machine for the constellation (link with the ground and among satellites).
- **Imager** – which allows getting realistic pictures according to current satellite position and attitude.

The following component has been added in the MCS VM:

- **Front-End** - It is the entry point of the MCS, handling the routing with Cosmos and the Standalone CryptoLib.
- **Mission** – it handles the scientific mission (see automatic mission section).

The following components have been adapted:

- **Standalone CryptoLib** : it is the library in charge of encryption for TC/TM messages and it is also responsible for adding the TC/TM Transfer Frame layer on top of the SPP layer. There is one execution instance per satellite
- **COSMOS** : it handles the communications (TC / TM SPP layer only) with the satellite.
- **NASA 42** : it handles flight dynamics and spacecraft visualization.

The chosen architecture for the communications in CCSDS (over UDP) between one Satellite VM and the MCS VM can be summarized by the diagram in Figure 7 (An example of IP addresses and ports for all the components is provided).

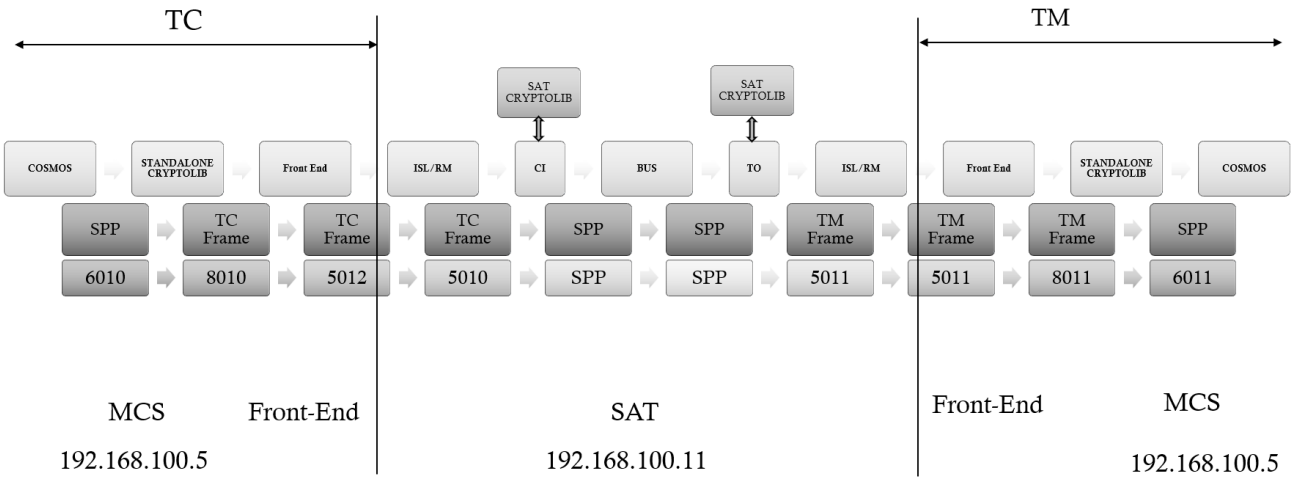


Figure 7. SAT-MCS Communications architecture.

It is important to observe that COSMOS and the satellite software bus keep working with SPP, while we can easily capture the traffic at Transfer Frame level between the Front-End component and the ISL/RM component. The transition SPP - TF is done

- on ground in standalone CryptoLib component.
- on board in CI / TO components (call to CryptoLib).

For comparison, one can see the SAT/MCS communications architecture for NOS3 v1.6.2 in Figure 8. One can see that NOS3 v1.6.2 is mainly working at SPP level.

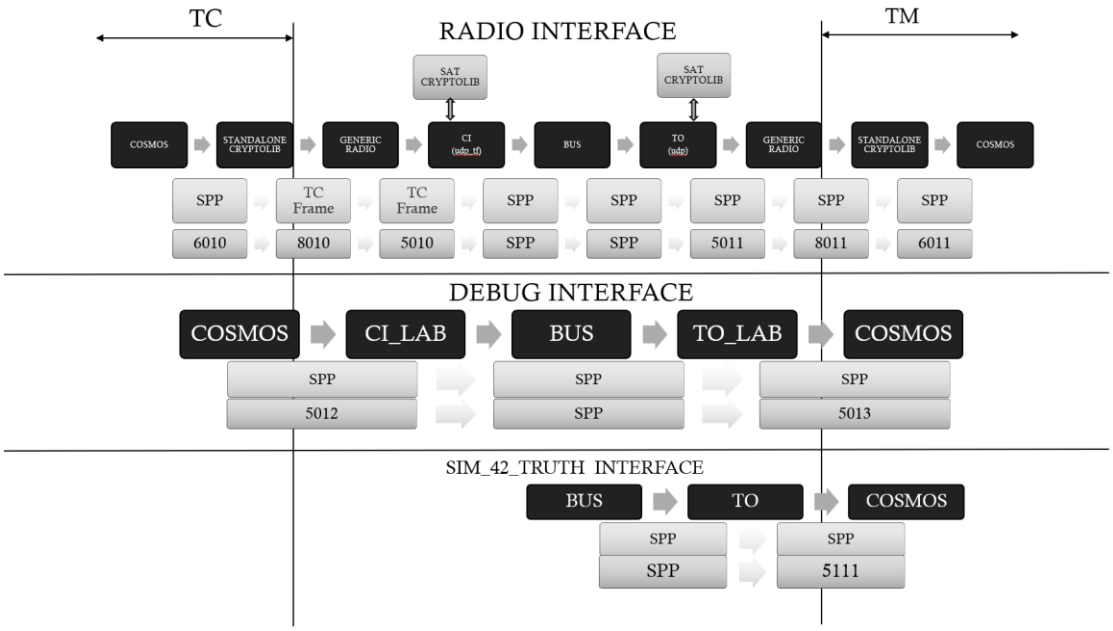


Figure 8. NASA NOS3 v1.6.2 MCS/SAT communications architecture.

For more detailed on NOS3 architecture and the components mentioned in Figure 8 one can have a look at NASA NOS3 Wiki at https://github.com/nasa/nos3/wiki.

A more general picture of the **simulator architecture is provided in Figure 9**, where a multi spacecraft configuration is presented, including an example of addresses and ports for each new component.

One can see that ISL/RM is the new entry point of the satellites VMs, while Front-End is the new entry point of the MCS VM. The Standalone CryptoLib and COSMOS are also duplicated for each new satellite to be handled into the constellation. NASA 42 is connected to all spacecraft using TCP sockets.

In the next sections, we will discuss more in details the proposed modifications to simulate a constellation of CubeSats.

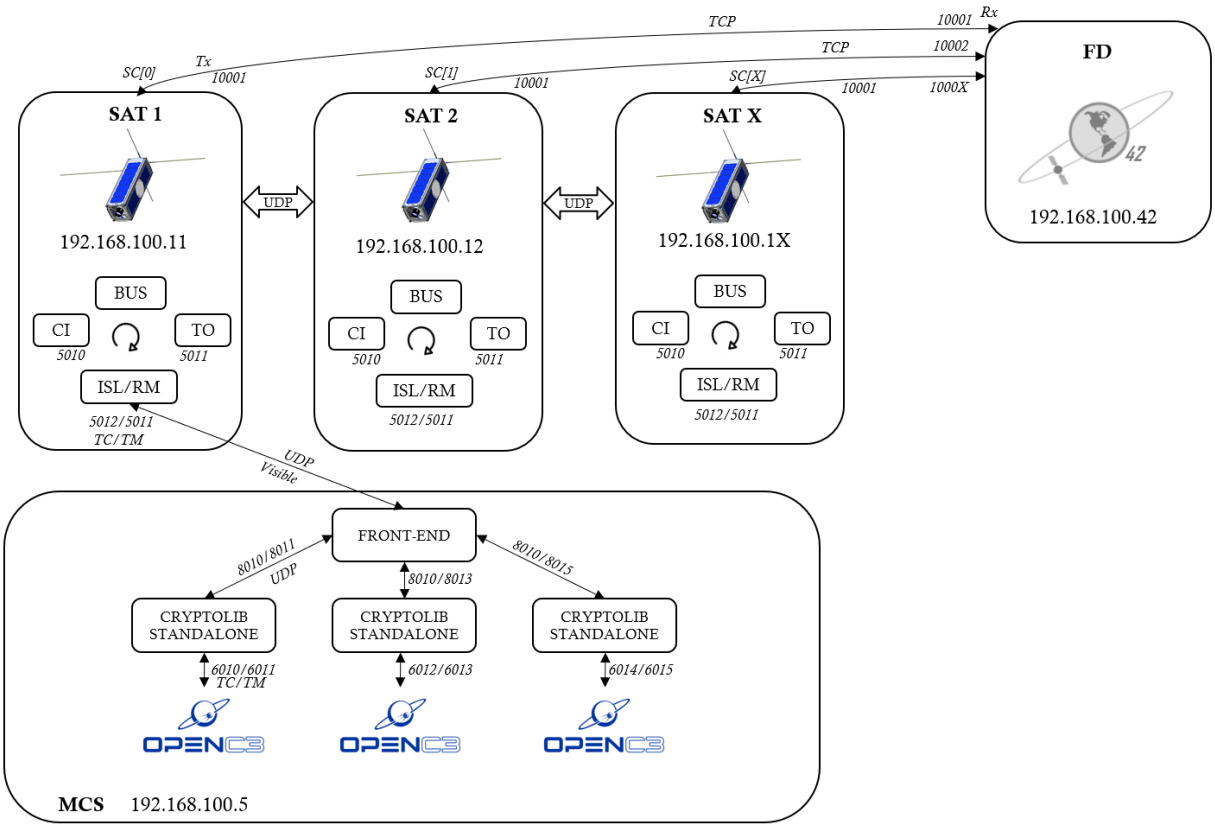


Figure 9. Overall architecture of the CubeSats constellation.

6 COSMOS

Cosmos (open C3) comes as a dockerized application, while CFS and 42 are directly executed on the host VM (NOS3 v1.6.2).

In particular, 8 docker containers are necessary for running one COSMOS MCS instance (see Table 1 and COSMOS documentation https://docs.openc3.com/docs).

Table 1. COSMOS Docker containers.

N	Name	Description
1	cosmos-openc3-cosmos-init-1	Copies files to Minio and configures COSMOS then exits
2	cosmos-openc3-operator-1	Main COSMOS container that runs the interfaces and target microservices
3	cosmos-openc3-cosmos-cmd-tlm-api-1	Rails server that provides all the COSMOS API endpoints
4	cosmos-openc3-cosmos-script-runner-api-1	Rails server that provides the Script API endpoints
5	cosmos-openc3-redis-1	Serves the static target configuration
6	cosmos-openc3-redis-ephemeral-1	Serves the streams containing the raw and decomutated data
7	cosmos-openc3-minio-1	Provides a S3 like bucket storage interface and also serves as a static webserver for the tool files
8	cosmos-openc3-traefik-1	Provides a reverse proxy and load balancer with routes to the COSMOS endpoints

By default, COSMOS can be used via its web interface at the address : http://localhost:2900

For the constellation deployment, it has been chosen to use 1 COSMOS instance (8 containers) for each satellite (see Figure 11). Therefore, the VM hosting the MCS will be running multiple COSMOS instances that can be accessed via specific ports : **2900, 2901, etc**. **Providing a web interface (tab) for each satellite**. This design choice is related to the operator comfort (separated web interfaces) and to the simple scalability of the docker application.

Indeed, looking at Figure 10, we can see that by simply specializing the “external targets”, the “user web browser” of COSMOS and the internal docker network it is possible to specialize COSMOS to different satellites. These configurations are managed via the *compose.yaml* file in the */opt/nos3/cosmos* folder (see Annexes).

WARNING: 8 Cosmos containers for each satellite means that MCS VM has to be provided with enough RAM to handle all the satellite in the constellation (see deployment requirements).

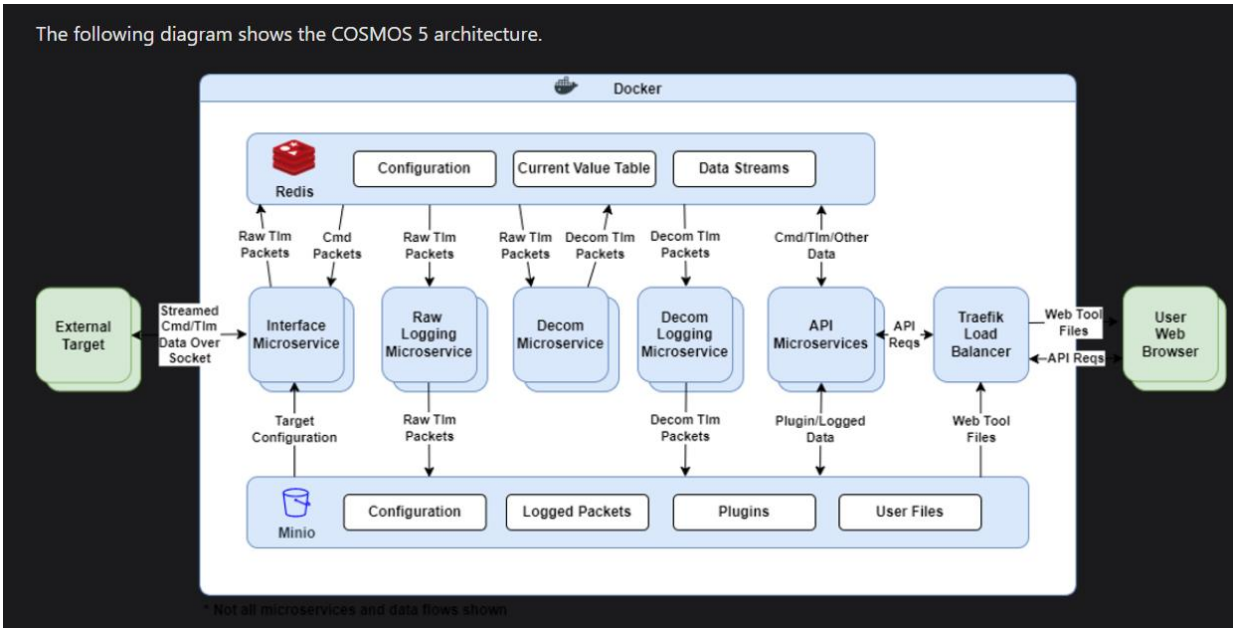


Figure 10. COSMOS Architecture.

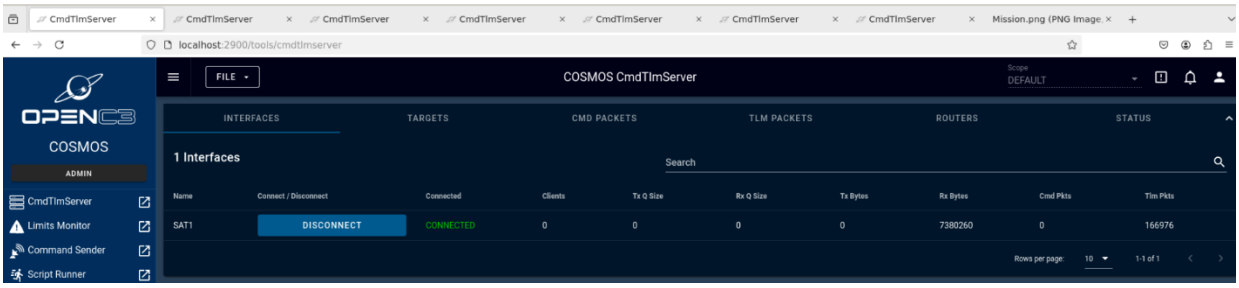


Figure 11. COSMOS Interface example. One web tab for each satellite in the constellation.

NOTE: In the current version of the simulator (04/26) the mission is automatically managed via the Automatic mission component (see Automatic mission section). However, it is important to note that the “Script Runner” menu of COSMOS can also be used to script a custom mission. In this case the user shall disable the automatic mission and define its own script in Cosmos (some examples are provided in TO_DEBUG folder via Cosmos GUI as in Figure 12).

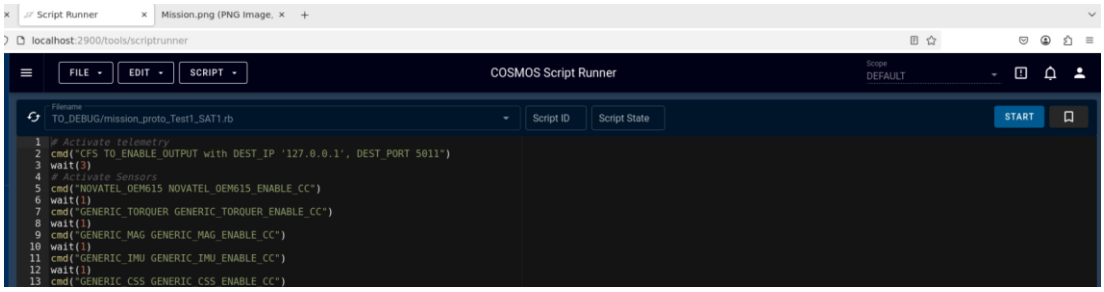


Figure 12. Example of Script Runner usage.

7 CryptoLib Standalone

Standalone library is used for its capacity to make the bridge between SPP and Transfer Frames and of course to eventually encrypt / decrypt the data.

The CryptoLib Standalone library (already available in the baseline v1.6.2 with version v1.2.2.0) has been adapted to use different ports according to the satellite to handle. Indeed, each instance accepts an input parameter, which is the id of the satellite to manage.

In Figure 13 one can see that a **separate CryptoLib tab is opened for each new satellite** (two in the example).

Figure 13. Modified CryptoLib for multi spacecraft configuration.

The CryptoLib (standalone) has also been modified to intercept some specific packets.

In particular,

- CFDP packets (see CFDP section).
- CAM App telemetry : to reassemble the images (.jpg) that are sent by the satellites (new images are saved in /tmp folder of VM MCS by default).
- IDS App Telemetry : to simulate operator reaction in case of malicious payload detected on board (see IDS section).

The user can activate/deactivate encryption in the CryptoLib tab by using *vcid 4* or *vcid 1* as in Figure 14 (change active TC virtual channel option).

```
cryptolib> vcid 4
Changed active virtual channel (VCID) to 4
cryptolib>
```

Figure 14. Example of encryption activation.

The security associations (SA) definition can be found in *sa_interface_inmemory.template.c* file. The user can observe that encryption is not active by default and that when activated the AES-GCM algorithm is used.

SDLS and SDLS-EP includes several procedures that are available via CryptoLib, as for example Over-The-Air-Rekeying (OTAR).

More info on SDLS and SDLS-EP (extended procedures) can be found here:

- https://ccsds.org/Pubs/355x0b2.pdf
- https://ccsds.org/Pubs/355x1b1.pdf

More info on CryptoLib known vulnerabilities:

- https://github.com/nasa/CryptoLib/security?page=1
- https://securitybynature.fr/post/hacking-cryptolib
- https://visionspace.com/crashing-cryptolib/

8 Front-End

Front-End (FE) is a new ground actor in charge of routing both TM and TC. In addition to ISL/RM component, it implements constellation features:

- Visibility
- Routing

At any moment it connects all “CosmosTF” and the spacecraft in visibility as shown in Figure 15.

DEFINITION: CosmosTF describes a couple composed of Cosmos (which handles SPP) and Standalone CryptoLib (which makes the bridge between SPP and transfer frames) as shown in Figure 15.

Front-End is based on Transfer Frames both in TC and TM.

As per configuration (see Figure 9) :

- FE receives TC on one port from all CosmosTF.
- any CosmosTF manages one satellite.
- FE receives TM on one port from satellites. Actually only satellites currently in visibility may send TM to FE.

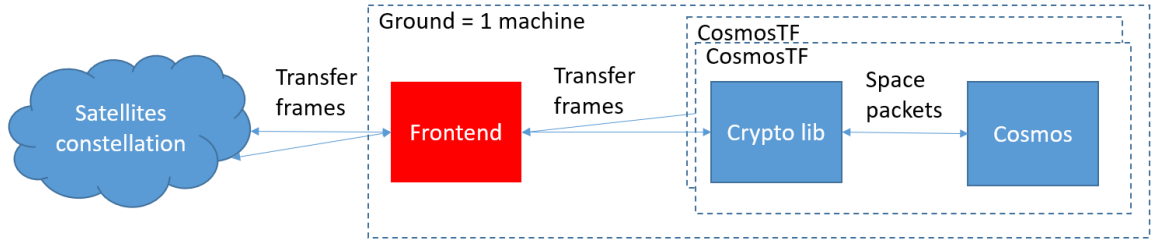


Figure 15. Front-End functional diagram.

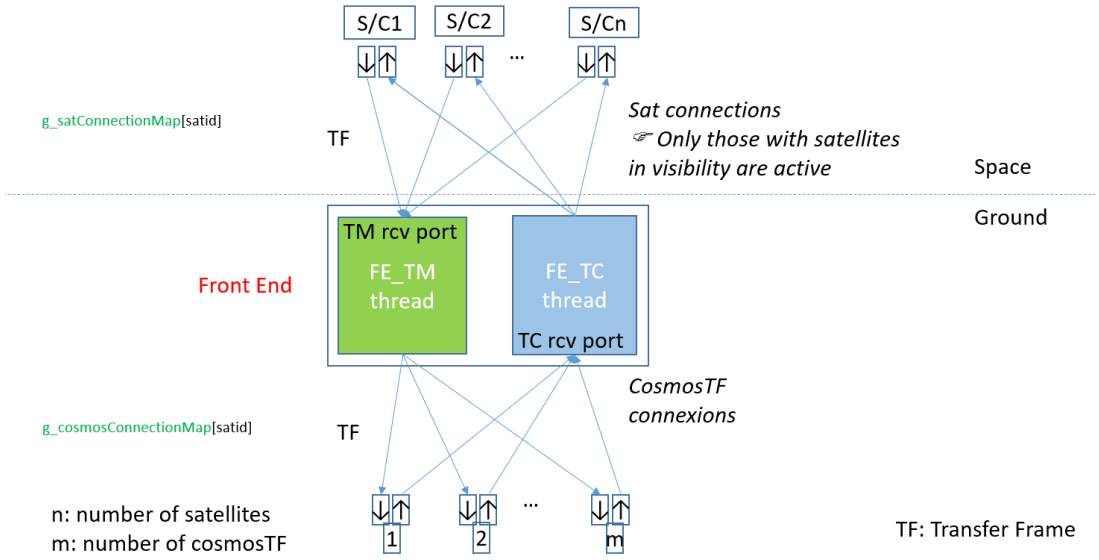


Figure 16. Front-End TM & TC threads and listening ports.

8.1 Interface between Front-End and CosmosTF

- **TC Side:** Each CosmosTF sends TC transfer frames to Front-End.
  Each TC transfer frames contains the destination satellite *(the one to be controlled)* in the TF header as per CCSDS standard. TC transfer frames sizes are variable to fit the exact needs for each TC.
- **TM Side:** Front-End sends TM transfer frames to CosmosTF corresponding to the origin satellite. The origin satellite is the satellite that produced the telemetry.
  Each TM transfer frames contains the origin satellite in the TF header as per CCSDS standard. TM transfer frames size is constant.

8.2 Interface between Front-End and Satellites

- **TC Side:** Front-End sends TC transfer frames to a « next » satellite according to the destination satellite. The next satellite may be the destination satellite if it is in visibility or an intermediate satellite that will allow to reach the destination satellite via ISL/RM (Inter Satellites Links/ Routing Machine). Each TC transfer frames contains the destination satellite in the TF header as per CCSDS standard.
- **TM Side:** Each satellite in visibility may send TM transfer frames to Front-End.
  Each TM transfer frames contains the origin satellite in the TF header as per CCSDS standard.

8.3 FE TC thread processing

FE listens for TC on a dedicated port for all cosmosTF.

On TC transfer frame reception from a cosmosTF:

- Retrieve the destination satellite from the TC transfer frame header.
- Determine the satellite to send the frame to. satToSendTo = route (ground, destination sat). This sat is mandatory in visibility.
- Retrieve the IP@ and CI port for this satellite (ip,port = f(sat#)).
- Forward the TC transfer frame to sat IP:port.

8.4 FE TM thread processing

FE listens for TM on a dedicated port for all satellites. Actually only sat in visibility may send TM transfer frame to FE.

On TM transfer frame reception from a S/C:

- Retrieve the Origin S/C from the TM transfer frame header.
- According to the Origin S/C, determines the CosmosTF to forward the frame to. CosmosTF = f (origin sat).
- Forward the TM transfer frame to the proper CosmosTF.

8.5 Front-End Network Configuration

Here an example of Front-End Network Configuration file.

```
//FE Configuration
//          TC rcv  TM rcv
PORTS   8010    5011
//                  IP                          CosmosTF      Sat id
//                                               rcv port     (list authorized)
COSMOS              192.168.100.105 8011    1
COSMOS              192.168.100.105 8013    2
COSMOS              192.168.100.105 8015    3
COSMOS              192.168.100.105 8017    4
COSMOS              192.168.100.105 8019    5
COSMOS              192.168.100.105 8021    6
COSMOS              192.168.100.105 8023    7
//
//                  Sat iD  Sat IP              Sat rcv
//                                              port (ISL)
SAT                 1       192.168.100.11  5012
SAT                 2       192.168.100.12  5012
SAT                 3       192.168.100.13  5012
SAT                 4       192.168.100.14    5012
SAT                 5       192.168.100.15    5012
SAT                 6       192.168.100.16    5012
SAT                 7       192.168.100.17    5012
```

8.6 Constellation Route Configuration thread

Front-End implements dynamic routing according to the real visibility of satellites. At any moment feeder links are established with the satellites in visibility of a gateway.

One consider Front-End as a unique ground endpoint virtually connected to many ground stations around the World. These ground stations are defined in configuration file for 42 (*Inp_Sim.txt*).

Along the simulation, routing tables are processed and sent to the constellation.
Satellite positions are anticipated in order to run in a “make before break” strategy: once a new satellite becomes visible and for more than one minute then its feeder is used to transmit the new routing table to the satellites.

In reality, in order to avoid a propagation module on ground, we rely on ISL component to propagate the orbit for next seconds (60) for a given satellite <i> and as if by magic, fills in the MCS /tmp directory the visi<i>.txt file. This file contains 1 or 0 according to the ground visibility of sat<i> in the next minute.

Only one feeder is considered at a time but the data structure would allow more than one.

Here is the structure of configuration for the constellation routing (see Figure 17). For information, the source map of FE is also provided in Annexes.

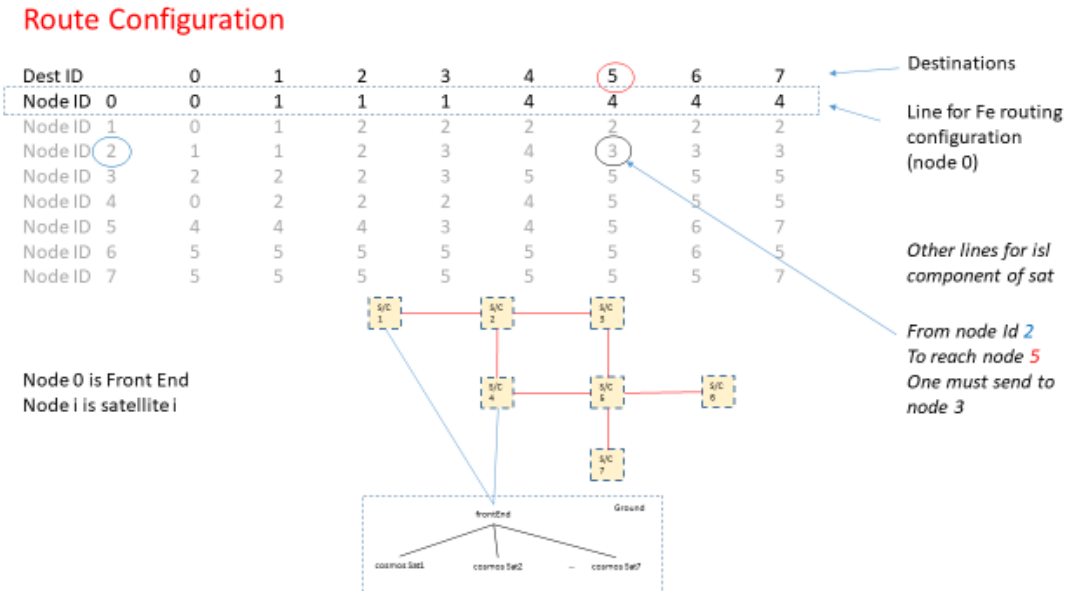


Figure 17. Constellation Route Configuration (not for the our mission).

In the frame of the exploration mission that we implemented, satellites are all spread over the same orbital plane. They form a ring, each having two neighbours. Altitude and Inclination of the orbit are 1300 km and 100.822° in order it to be sun-synchronous (see Figure 18).

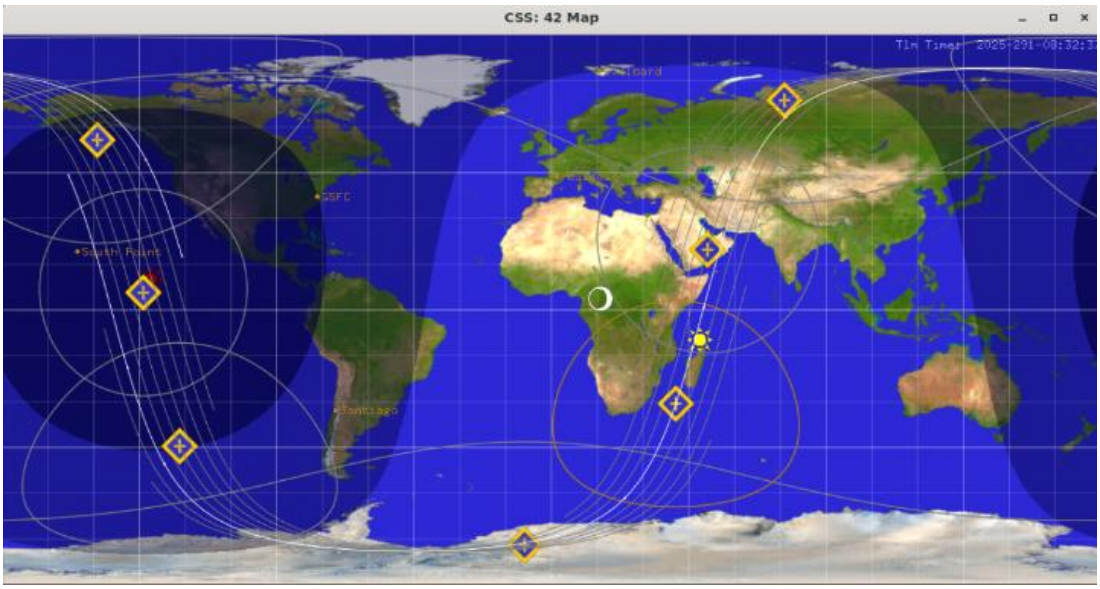


Figure 18. Example of mission (7 satellites on sun-synchronous orbit).

Front-End also use watchdogs to check that the feeder links are really connected. Watchdog are check in both sides (uplink: FE sends and ISL receives, downlink: ISL sends and FE receives)

9 ISL/RM – Inter Satellite Link / Routing Machine

ISL/RM is a new component in NOS3 architecture. It implements a Routing Machine, Inter Satellite Links and Feeder links *(ground visibility feature)*. It also simulates simplified radio connections under UDP for ISL and Feeder links. Communication protocol is SPPs in Transfer frames over UDP. Currently, routing and network configurations are dynamically processed according to visibility. ISL links are currently stable for the observation mission; Sat are all on the same orbit plane, spread around the World.

In Figure 19 an overview of ISL/RM implementation is presented. The ISL/RM architecture is detailed in Figure 20.

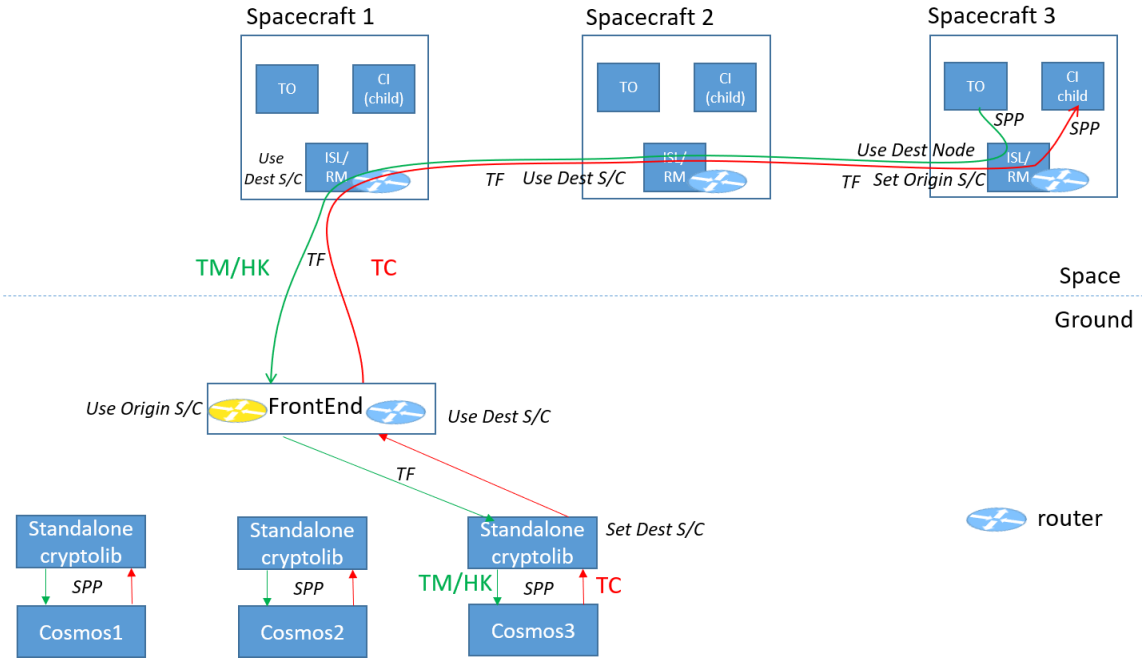


Figure 19. ISL Overview.

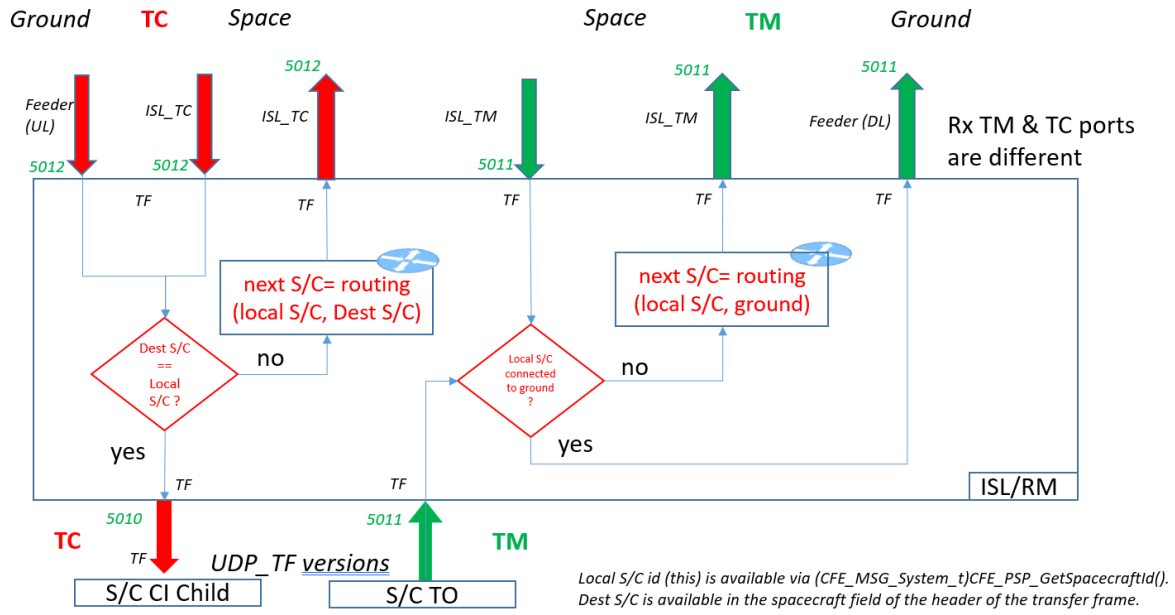


Figure 20. ISL/RM component architecture.

9.1 TC Routing

ISL/RM routes TC transfer frames coming from

- Front-End
- Or another satellite' ISL/RM

ISL sends TC transfer frames to

- CI if the destination satellite is the local one
- Or to another satellite ISL/RM according to route table

TC Routing is a function that determines the Next node given

- ➢ The Local S/C
- ➢ The Destination S/C
- ➢ The Next node is a S/C or CI

TC Routing is implemented in ISL/RM component.

9.2 TM Routing

ISL/RM route TM transfer frames coming from

- TO
- Or another satellite ISL/RM

ISL sends TM transfer frames to

- Front-End if the satellite is in visibility of Front-End

- Or to another satellite ISL/RM according to route table

TM Routing is a function that determines the Next node given

- The Local S/C
- The Destination which is « Ground »
- The Next node is a S/C or FE (ground)

TM Routing is implemented in ISL/RM component.

Geometric Visibility is computed by ISL when the GPS data are available (NB: NOVATEL GPS activation by TC is required) the data are then transferred to MCS VM in /tmp folder. Indeed, the ISL computes current and forecasted visibility and create a Boolean for the ground station in */tmp/visiX.txt.* The user can check visibility Booleans for all satellites in the constellation by simply typing in /tmp of MCS VM :

cat visi*.txt.

This is a workaround to provide satellite visibility information to ground station as there is no direct communication between MCS VM and NASA 42 VM (This could be improved in further releases).

The ground station used for visibility computation are the one presented in NASA 42 2D Map and defined with flag TRUE in the file *Inp_Sim.txt*.

WARNING: If there is no visible ground station for all the satellites in the constellation then the packets will be **dropped by ISL** (this can be improved in further releases).

9.3 ISL watchdog handshake

ISL/RM implements watchdog handshake throughout Inter Satellite Links and Feeder Links. Watchdog handshake for any link is bi directional.

- ISL sends a watchdog on every connected link any seconds
- ISL expects receiving a watchdog on every connected link at every seconds

In case of error, the corresponding link is considered in failure.

```
EVS Port1 1/1/ISL 51: the link between sat 1 and sat 2 is up
EVS Port1 1/1/ISL 51: the link between sat 1 and sat 0 is up
EVS Port1 1/1/ISL 51: the link between sat 1 and sat 7 is up
```

Figure 21. example of watchdog output. link is up = nominal, link is down = error.

10 NASA 42 constellation

The FD (Flight Dynamics) VM will run only NASA 42, that is already capable of managing multi spacecrafts simulations. The 42 “InOut” (https://github.com/ericstoneking/42) folder has been modified in order to connect multiple spacecraft to 42.

In particular, the *Inp_Sim.txt* file is modified to consider a multi spacecraft configuration (from SC[0] to SC[N]).

An example of NOS3 sockets configuration for 42 is described in Table 2. This configuration has to be multiplied for the number of spacecraft (SC[0], SC[1], … SC[N]) in the Inp_IPC.txt file.

The Tx and Rx folder have also been modified in order to connect properly 42 to the spacecraft into the simulation (see for example Figure 9).

It is important to note that 42 has no information about the spacecraft ID or NORAD name of the satellite, thus it is important to properly connect the SC[index] with the correct spacecraft before launching the simulation.

An example of 2 CubeSat configuration in 42 Cam is shown in Figure 22. More details on NASA 42 multi spacecraft configuration can be found in the Annexes.

The user can customize the input sockets via the *Inp_IPC.txt* file, but it is also possible to modify the sockets directly in 42 source code. An example is provided in Annexes.

If the user need to modify the sockets, it is suggested to use *Inp_IPC.txt* files in *Install_tools_nos3/Specialize_42_InOut* folder (see Deployment and Development sections).

	Tx	Rx
RW0		4278
RW0	4277	
RW1		4378
RW1	4377	
RW2		4478
RW2	4477	
Torquer		4279
IPC4	4245	
CSS	4227	
MAG	4234	
FSS	4281	
IMU	4280	
ST1	4282	
ST2	4283	

Table 2. Example of NOS3 / 42 sockets configuration (Inp_IPC.txt).

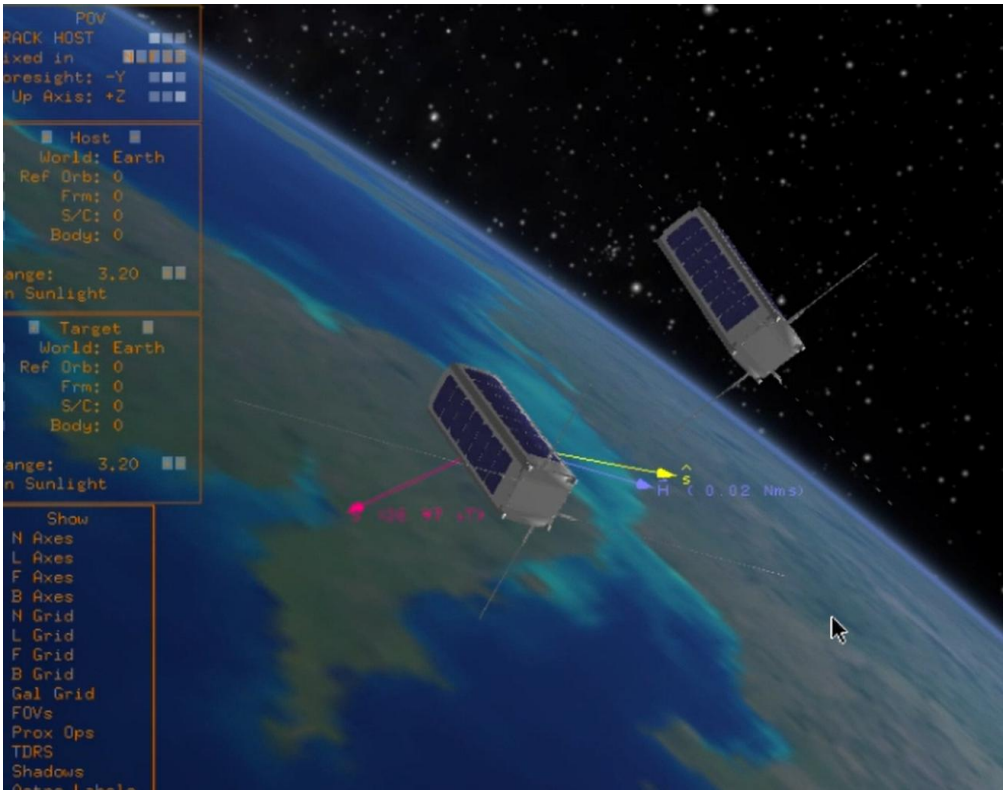


Figure 22. Example of 42 Cam with 2 CubeSats.

11 ADCS “Normal Mode”

A pseudo “normal mode” has been introduced in CSS simulator to be able to track a target on Earth based on Latitude and Longitude.

This tracking mode is based on GPS and Star Tracker information → that **shall be activated by TC before switching to normal mode**.

It is important to note that when normal mode is activated the satellite will start pointing to Nadir by default.

The target position is provided by the ground via TC (SET_TARGET command), while the satellite position and attitude are obtained via GPS and Star Tracker (ST) components. The periodic call to GPS and ST components can be verified in the scheduler tables in */fsw/nos3_defs/tables.*

The attitude information (quaternion) from ECEF to body frame (called *qbw*) is not present by default in NOS3 and we have added a specific socket in NASA 42 (see *42/Source/42sensors.c* and *42/Source/IPC/SimWriteToSocket.c*) to provide this information to the ADCS.

The result is that the ADCS is provided periodically with the following information:

- Satellite position in ECEF frame
- Satellite attitude from ECEF to body frame
- Target Latitude and Longitude

Based on these information a simple PD (or PID) controller is used to align the desired target direction with the -Y body axis of the satellite (we consider the hypothesis that the camera payload is on -Y face of the satellite).

In order to simplify tuning and adaptations, the controller gains are defined as a function of the expected satellite time response. The controller can be tuned/modified/replaced in *generic_adcs_adac.c* where the ADCS modes are defined. Some highly simplified prototypes of estimators for GPS and ST measures are also introduced in *generic_adcs_utilities.c*.

As the ADCS component manage position and attitude information, the component is used also to pass information to the “Imager” component. A *coord.txt* file is regularly updated in */tmp* in order to provide to the imager the necessary information to display the updated view from the onboard camera. From a qualitative point of view, the user can visually verify the pointing accuracy and stability on the imager tab. Moreover, one can compare the results wrt NASA 42 camera in body frame display (but considering that FOV are different). An example is provided in Figure 23. It is important to note that by default **the Imager is updated only when the ADCS is in Normal Mode.**

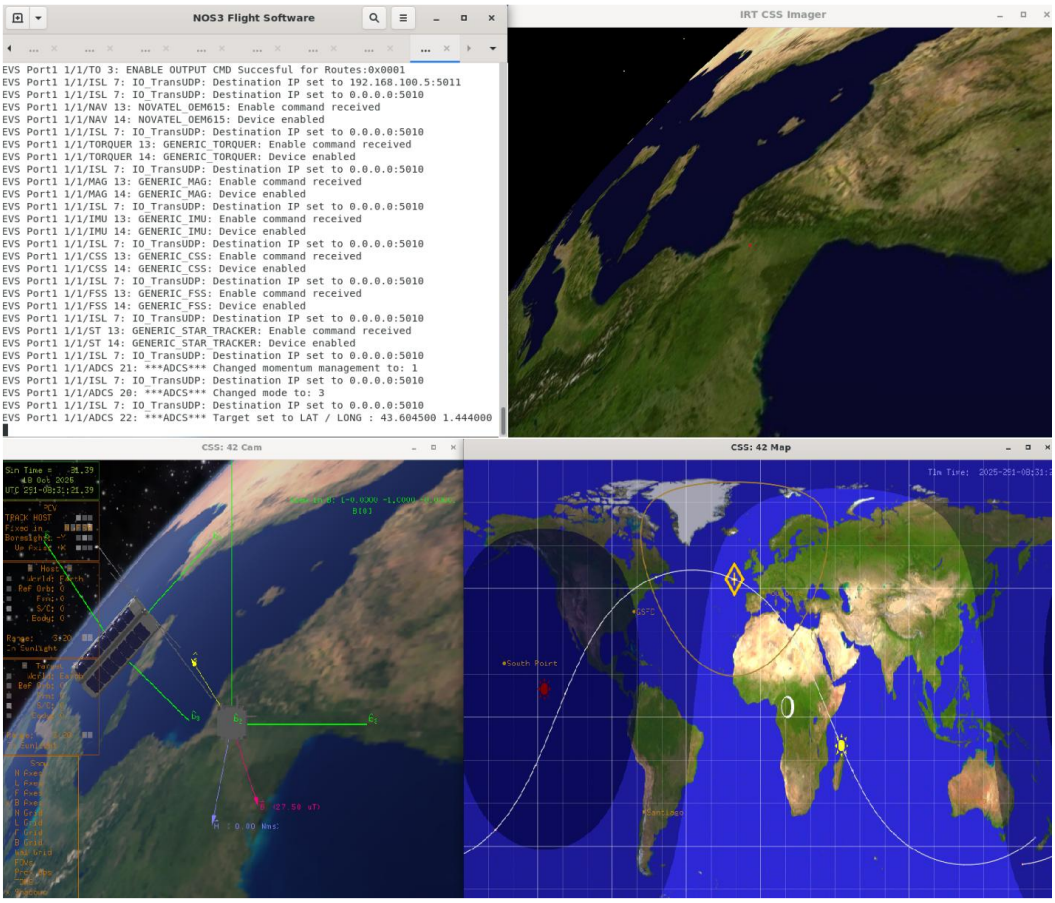


Figure 23. Imager (top right) vs NASA 42 camera (bottom left).

WARNING: GPS and ST information received via NASA 42 sockets can freeze or be unavailable (NaN), in particular during simulation start. This can cause pointing errors or erratic behaviours during the simulation. Currently the simulator is very simple concerning ADCS modes (it does not include mode reconfiguration, fault detection, robust estimation, etc…). Please do not hesitate to improve the ADCS code as a function of your needs and if you observe abnormal controller behaviours please verify the input data.

WARNING: quaternions from NASA 42 are in order X-Y-Z-W. Check also star tracker mounting angles when using the data for your application in the ADCS. By default they are set to zero in *sims/cfg/InOut* folder.

WARNING: tuning stability and performance of the controller have not been verified ! This is a simple prototype to close the mission loop (the goal is to perform **cybersecurity testing** and not accurate pointing!). **Do not hesitate to improve the ADCS controller as a function of your needs**. In the current release, the controller does not take into account any frequency filtering to anticipate satellite flexibility. Indeed, by default the satellite is rigid. However, a priori there is the possibility in NASA 42 to play with flexibility, in this case the user will have to adapt the controller (if required) to avoid instability.

12 Imager

The imager is an external simulation tool with two main functions:

- to visualize in near-real time what the camera “sees” based on the satellite’s position and attitude; On thus view the target appears as a red point.
- to obtain realistic photos when a photo capture command is sent to the Arducam (NOS3 optical payload).

Communications with imager are done via 3 files:
> /tmp/coord.txt (in)
> /tmp/picreq (in)
> /tmp/image.jpg (out)

As input, it receives information about the satellite’s position and attitude in ECEF frame, the coordinates of a target on Earth to display it in continuous visualization, and the position of the Sun (coordinates are passed in */tmp/coord.txt* file) . As output, on request (presence of file /tmp/picreq), it produces an image (in /tmp/image.jpg) showing the 3D scene as viewed from the onboard camera. The imager is based on GPU technology, which allows modelling the Earth as a sphere with a texture applied to it, and the 1,000 brightest stars and their respective magnitudes.

The user can modify the main image used by the Imager to create Earth texture and the list of available stars directly in the main folder of the component : */home/nos3/eclipse-workspace/imager.* An example of Imager output is shown in Figure 24.

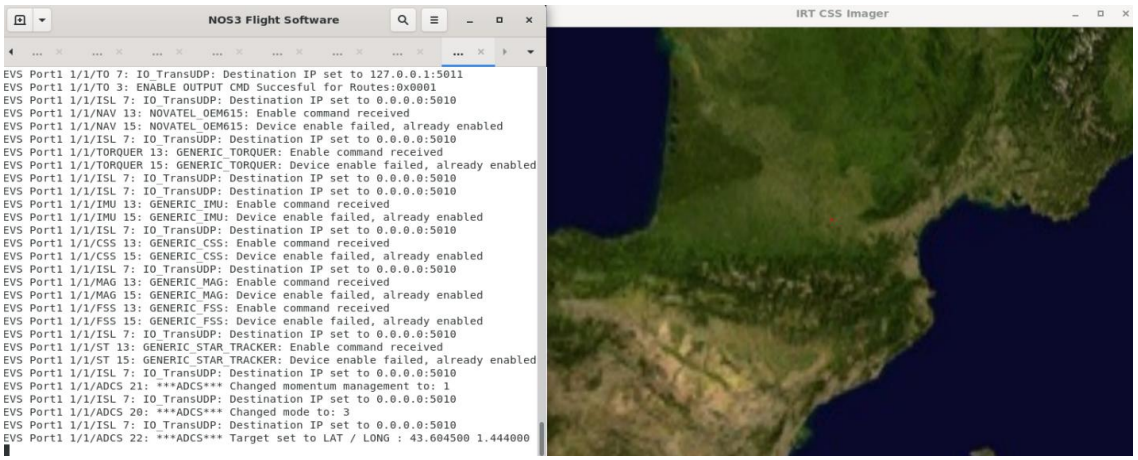


Figure 24. FSW and Imager output. Target is Toulouse.

NOTE: The eclipse phase is simulated using the scalar product between the sun position and the Earth normal vector. This is done in the shader3d.vs file in shader directory.
We consider max (sqrt(sqrt(normal . lightPos)), 0.2) as the shadow factor. That means that during eclipses 20% of the light is still available. The double sqrt allows to sharpen the transition.

NOTE: /tmp/coord.txt content:
- 3 double for camera position in ECEF (x y z)
- 3 double for satDirX vector in ECEF (x y z)

- 3 double for satDirY vector in ECEF (x y z)
- 3 double for satDirZ vector in ECEF (x y z)
- 3 double for Sun position in ECEF (x y z)
- 1 double for raan angle (angle) *used to draw stars.*

NOTE: To modify the main image used by the imager, the user can replace the "***Earth.jpg"*** image in the component folder (NB: image pixels needs to be a power of 2).

The target selected by the satellite is shown with a red dot on the main picture.

13 Dynamic Routing

The simulator handle a simple case of dynamic routing : the satellites are all on the same sun synchronous orbit (simple ring topology with 1 feeder) and the routing tables are updated regularly according to visibility.

The visibility is provided by */tmp/visiX.txt* files in VM MCS. The user can verify visibility by simply using ***cat visi*.txt*** in /tmp folder of VM MCS.

The visibility (feeder link) is updated when one satellite in the constellation pass from non visible to “stably” visible. Stably means in this case that the satellite is visible by at least 1 ground station and it will stay visible for at least a given amount of time (ex. 60sec). Moreover, the feeder link is not updated if it was updated “shortly” before (predefined time defined in Front-End thread).

In this way the feeder link is updated regularly but avoiding too rapid changes of the link.

The ISL are updated to consider the shortest path to the ground station in a ring topology.

The topology (simple ring topology with 1 feeder) is defined in the function ***FE_routegen.cpp***, the user can modify the topology here if needed.

In the same function (FE_routegen.cpp), one can see that for every route update the following commands are sent by Front-End (for each satellite):

- Ask CFDP uplink of the new routing table via CryptoLib (see CFDP section).
- Update ISL configuration. The priority is to the satellites closer to the new feeder link.

An example of FSW output during dynamic table reconfiguration is shown in Figure 25 and Figure 26.

```
EVS Port1 1/1/CF 110: Received packet of length 53 and sent to CFDP server
EVS Port1 1/1/CF 110: Received packet of length 250 and sent to CFDP server
EVS Port1 1/1/CF 110: Received packet of length 23 and sent to CFDP server
```

Figure 25. FSW received the new table.

```
EVS Port1 1/1/ISL 20: Configuration command received for file: routeConfig.txt
EVS Port1 1/1/ISL 20: Route configuration for Sat 1 :
0 1 2 2 2 7 7 7
```

Figure 26. ISL updated table configuration.

14 Automatic Mission

In the folder eclipse-workspace/mission an automatic mission is defined for a constellation of satellites with optical payload. When launching the simulator via

```
./start.sh
```

the automatic mission will perform the following actions:

- wait for some time for the simulator to be ready (launch all process on all VMs)
- initialize all the satellites via TCs (enable sensors, enable telemetry and switch to “Normal Mode”)
- wait for some time for all the satellites to be in Normal mode (Nadir pointing at start) and ready to start the mission
- launch the scientific mission (capture photos of predefined targets when a satellite is within a predefined distance)

The automatic mission targets are defined in ***eclipse-workspace/mission/config/targets.txt*** file. The MCS operator can follow the automatic mission evolution for example by looking at ***Mission.png*** image in ***eclipse-workspace/mission/src***. Indeed, the captured images are progressively overlaid to the targets presented on the map as shown in Figure 27 (**refresh Firefox tab to see new targets**).

The automatic mission will iterate on the list of targets defined in *targets.txt* file. If the target photo is captured correctly, then the associated satellite is free and it can be used to point to another target. The capture is expected to take less than a predefined time defined in mission main code, in case of issues (too much time to take the photo) the satellite is free and we wait for another satellite to capture a photo of the target. When the photo capture is completed and the satellite is free, pointing is set to nadir (via the default values (0,0) for latitude and longitude).

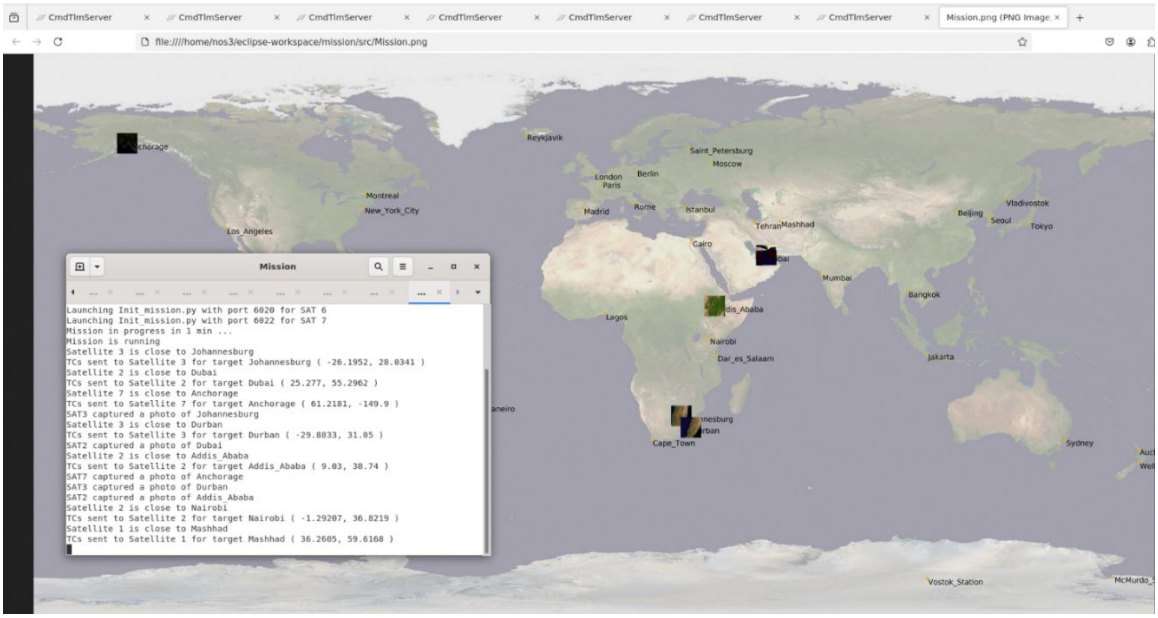


Figure 27. The MCS operator can follow the mission (photo captured vs targets).

15 Scenario Manager

The user has the possibility to generate specific scenarios and datasets via the Scenario Manager component or via the simulation orchestrator in css-dataset repository.

The scenario manager component is introduced to automate the generation of data based on CSS simulator. Indeed, the user has the possibility to define a simulation batch that include several attack scenarios and initial conditions and later to store the .pcap and logs for each run. **NOTE:** This process is largely improved in css-dataset orchestrator and **we suggest to use directly the css-dataset repository** to generate data. However, the scenario manager repository is provided as an example.

The scenario manager is a simple set of scripts that helps in the automatic simulation of scenarios. In order to launch a single scenario, the user can call (from VM SAT1 and a terminator tab):

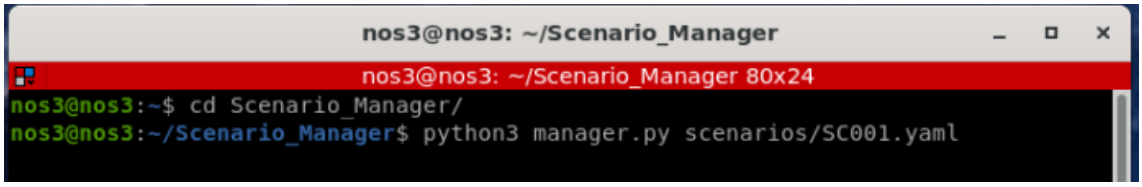


Figure 28. Call to manager.py

with the scenario characteristics defined in SC001.yaml.

The script Manager.py will sequentially:

- Use the list of targets defined in SC001.yaml for the automatic mission
- start the simulation
- start a tcpdump in /tmp of VM MCS
- launch the attacks defined in SC001.yaml (according to predefined times)
- stop the tcpdump
- stop the simulation.
- save relevant files in /tmp of VM MCS

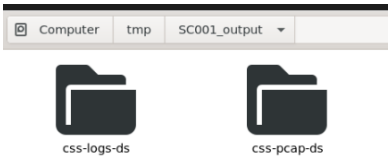


Figure 29. Example of output for a simulation generated with manager.py.

In order to launch multiple scenarios in a sequence, the user can for example define a simple bash script (as for example *./CSS_Test.sh* that can be used to launch 2 scenarios in a sequence). In this way the Manager.py will be called multiple times and multiple datasets will be generated automatically. It is highly recommended to perform:

```
nos3@nos3:~/Install_tools_nos3$ ./Do Make All VM.sh
```

Between 2 simulations in order to avoid issues. In particular when simulating attacks that modify the “build” folders of NOS3. Please check the README.txt file for more details.

16 Attack library

The user can select several attack scenarios via the dedicated folder CSS_Attacks.

In particular, by typing in a gnome terminal

```
./Master_Attacks_Scripts.sh 0
```

the user can display all the available attacks (a summary is presented in Figure 30).

```
./Master_Attacks_Scripts.sh <number of the exploit>
```

will launch a specific attack by launching a specific bash or python script that correspond to the exploit.

The attacks are described inside the code and also in the publications associated to CSS research project (see **Publications**).

Some attacks are based on known, previously published vulnerabilities (see for example Figure 31, and the A6 or A22 attacks). Others were designed in the scope of this project.

N	Attack Name	Description	Scenario
A1	CI KILL	Send TC to kill CI of SAT X (Sabotage-DoS)	2.3
A2	ISL KILL	Send TC to kill ISL (Sabotage-DoS)	2.3
A3	APPS KILL	Send TC to kill several cFS Apps (Sabotage-DoS)	2.3
A4	FE KILL	Send command to kill Front End on ground (Sabotage-DoS)	5.3
A5	CAM KILL	Send TC to restart CAM App while taking picture (FSW Crash - Sabotage)	2.3
A6	CRYPTO TC CRASH	Send TC to crash CryptoLib on board (DoS)	2.3
A7	TC SB FLOOD	Send TC continuously to flood Software Bus (SB) from ground (DoS)	5.3
A8	APP DELETE	Delete one APP via TC plus the associated .so file in /cf (Sabotage)	2.3
A9	CAM GET	Intercept Camera Payload Data on ground (Confidentiality)	6.3
A10	CAM CORRUPT	Rewrite Camera Payload Data on board (Confidentiality - Sabotage)	5.11
A11	EPS SABOTAGE	Send TCs to discharge the battery via EPS switch all ON plus ADCS sabotage (Integrity)	2.3
A12	ADCS EVIL TC	Send TC to put satellite in rapid rotation around the 3 axis (Sabotage - Integrity)	2.3
A13	EVIL APP FLOOD	Send TC to load malicious App (.so) that flood the SB (DoS)	2.2
A14	CI KILL from APP	Send TC from evil CAM APP to KILL CI (Sabotage-DoS)	2.2
A15	FILES DELETE from APP	Send delete all to /cf from CAM app (Sabotage-DoS)	2.2
A16	SCID FE INV	SCID Inversion in Front End (Sabotage)	5.10
A17	SCID FE DOUBLE	SCID Duplication in Front End (Sabotage)	5.10
A18	ISL AUTOLOOP	ISL route tables modification via TC inducing autoloop (DoS)	5.10
A19	ISL LOOP	ISL route tables modification via TC inducing a multi satellites loop (DoS)	5.10
A20	CRYPTO BYPASS	Bypass SDLS by using CryptoLib vulnerabilities (Sabotage - Takeover)	2.3
A21	CRYPTO HIJACK	Load a new encryption key (Takeover)	4.1
A22	FULL CRYPTO HIJACK	Full hijacking procedure based on OTAR PDU commands (Takeover)	4.1
A23	ISL KILL from APP	Send TC to kill ISL from evil app (Sabotage-DoS)	2.2
A24	CAM KILL from APP	Send TC to restart CAM App while taking picture from evil app (FSW Crash - Sabotage)	2.2
A25	APP DELETE from APP	Delete one APP via TC plus the associated .so file in /cf from evil app (Sabotage)	2.2
A26	ADCS EVIL TC from APP	Send TC to put satellite in rapid rotation around the 3 axis from evil app (Sabotage - Integrity)	2.2
A27	FILES DELETE	Delete all configuration and library (.so) files (Sabotage - DoS)	2.3

Figure 30. List of available attacks (as of June 2025).

It is important to note that the list in Figure 30 represent only a small sample of the attacks that can be developed on the platform based on existing vulnerabilities. For example, one can use the vulnerabilities identified by Visionspace (see Figure 31) to develop other attacks

or simply use the intrinsic vulnerabilities of cFS architecture (ex. unencrypted software bus with application).

0-Days Discovered in Space Software

Product	CVE	Severity
NASA cFS Aquila	CVE-2025-25371, CVE-2025-25372, CVE-2025-25374	HIGH
	CVE-2025-25373	CRITICAL
NASA Cryptolib 1.3.0	CVE-2024-44910, CVE-2024-44911, CVE-2024-44912	HIGH
NASA fprime v3.4.3	CVE-2024-55029	MEDIUM
	CVE-2024-55028, CVE-2024-55030	CRITICAL
OpenC3 Cosmos	CVE-2025-28380, CVE-2025-28381, CVE-2025-28382, CVE-2025-28384, CVE-2025-28386, CVE-2025-28388, CVE-2025-28389,	TBD
NASA AIT-Core 2.5.2	CVE-2024-35057, CVE-2024-35058, CVE-2024-35059, CVE-2024-35060, CVE-2024-35061	HIGH
	CVE-2024-35056	CRITICAL
SAS Yamcs 5.8.6	CVE-2023-45279, CVE-2023-45280, CVE-2023-45281, CVE-2023-46470, CVE-2023-46471, CVE-2023-47311	MEDIUM
	CVE-2023-45277	HIGH
	CVE-2023-45278	CRITICAL
NASA Open MCT 3.1.0	CVE-2023-45884, CVE-2023-45885	MEDIUM
	CVE-2023-45282	HIGH

Figure 31. Extract from Visionspace presentation at CYSAT 2025 by Milenko Starcik.

From cybersecurity demonstration/testing point of view, the attacks defined in Figure 30 enable us to show different type of exploit impacting confidentiality, integrity or availability. Thus, they are an interesting set to start investigating or training on space system cybersecurity.

WARNING: some of the attack consequences might not be fully realistic (or they require further development of the simulator to be realistic).

Example 1): the attack 12, inducing rapid rotation around the 3 axis, requires further development to be fully realistic. Indeed, in the current version, the communications are maintained even after the attack. However, in reality, the satellite should experience antenna pointing issues and thus complete or partial loss of the communications.

Example 2): the attack 11, impacting the battery integrity, requires further development to be fully realistic. The battery voltage after the attack is decreased, but the voltage dynamic and values are not realistic in the current version. **The EPS model shall be modified to better represent the voltage dynamic** and the impact of switches and systems on battery voltage.

Example 3): flooding attacks (from ground or via malicious applications) shall be tuned (number of messages and delays) to represent a real scenario according to available resources.

NOTE: for data training generation purpose, attacks are generally labelled using an unused field of SPP or TF. For example, for attacks based on SPP, we used the checksum field

(unused in NOS3). This label is reset to 0 when the message reaches the Software Bus, before being sent to its destination.

The following table shows the considered labels and their value.

Attack	Label (where)	Value
A1	SPP (8th octet)	0x0B = 11
A2	SPP (8th octet)	0x0C = 12
A3	SPP (8th octet)	0x0D = 13
A4	N/A	N/A
A5	SPP (8th octet)	0x0F = 15
A6	TF(6th bit of first octet)	6th bit of 1st octet to 1
A7	SPP (8th octet)	0x11 = 17
A8	SPP (8th octet)	0x12 = 18
A9	N/A	
A10	N/A	
A11	SPP (8th octet)	0x15 = 21
A12	SPP (8th octet)	0x16 = 22
A13	only SPP messages from malicious cam	0x17 = 23
A14	only SPP messages from malicious cam	0x18 = 24
A15	only SPP messages from malicious cam	0x19 = 25
A16	N/A	
A17	N/A	
A18	SPP (8th octet)	0x1C = 28
A19	SPP (8th octet)	0x1D = 29
A20	TF(6th bit of first octet)	6th bit of 1st octet to 1
A21	TF(6th bit of first octet)	6th bit of 1st octet to 1
A22	TF(6th bit of first octet)	6th bit of 1st octet to 1
A23	only SPP messages from malicious cam	0x21 = 33
A24	only SPP messages from malicious cam	0x22 = 34
A25	only SPP messages from malicious cam	0x23 = 35
A26	only SPP messages from malicious cam	0x34 = 36
A27	SPP (8th octet)	0x35 = 37

Table 3. Attacks labelling.

Some attacks use an edited version of the Camera (payload of the satellite) which is malicious. For these attacks, the source code is located under the *CSS_Attacks/Malicious_cams_source* folder.

If someone wanted to experiment with or edit such attacks, the first step would be to pick the malicious source code and place it in the *components/arducam/fsw/src* folder, renamed as the *cam_app.c* file. Then, the user can use **make clean && make** to generate the appropriate .so file (located under *fsw/build/exe/cpu1/cf/arducam.so*).

The .so file can then be copied to the CSS_Attacks folder and later uploaded to the satellite via ***./Attack_Cam.sh <file_name>.***

17 Intrusion Detection System

It is important to note that IDS/IPS component is active by default, you should modify the script */github-nos3/components/ids/fsw/src/ids_probes.*h to disable the 3 probes and recompile the simulator. The IDS is in mode "prevention" by default once activated. Logs about attacks are dropped in /tmp folder.

The IDS module embedded in the satellite uses a multilayer approach with a set of probes (in different locations of the architecture), and a set of detection modules. Three specific probes were designed:

1) The first probe is located at the arrival of each TC in the satellite (i.e. in the ISL/RM component), and aimed at capturing features of the network traffic exchanged between the satellite and the ground segment: header data, packet inter-arrival time, size of packets, etc. As we assume that cryptography is enabled, this probe is not able to get the content of the packet and thus can only get some characteristics at the network flow level and access the packet header.
2) The second probe is aimed at reporting information on deciphered TCs coming from the ground, and not sent yet on the Software Bus, to distinguish them from internal messages. This probe is located into the CI component on the NOS3 platform. Both command type and command parameters are captured.
3) The third probe is aimed at capturing information in the same way as the second one, but is located on the Software Bus. This positioning allows to gather information related to the internal exchanges of the satellite between every component, and thus would enable the detection of a malicious payload. This probe is currently implemented by modifying the software bus itself in order to capture all messages exchanged on the bus along with the source of the message.

Information gathered by these probes is used as an input to the intrusion detection and prevention algorithms. A representation of the location of the probes is given in Figure 32. It is important to note that, to have defence mechanisms as accurate as possible, a standard model of ‘normal’ commands sent from the ground representative of real mission operation is mandatory. In this project this is handled by the Automatic Mission utility, but one could imagine editing the mission behaviour depending on the needs.

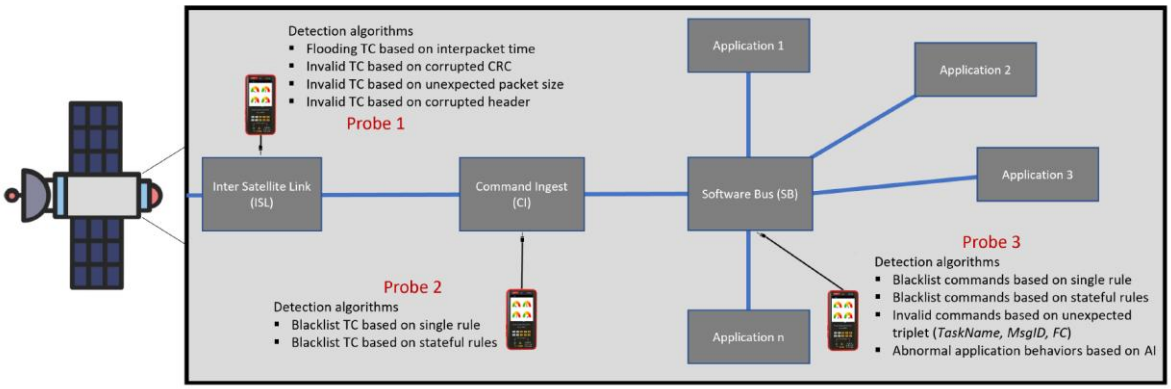


Figure 32. IDS Architecture.

The IDS/IPS algorithms are located in the *components/ids/fsw/src* folder. These algorithms include basic detection heuristics (used with the input of Probe 1 & Probe 2), and rule-based approaches (Probe 2 & Probe 3).

The principal component of the rule-based detection is the TC Analyzer, which code is located in the *ids_analyzer.c* file. Its behaviour is simple; it takes as input a set of detection rules (which resemble firewall rules, but in a blacklist approach) that can be customized by the user, and a packet. If the packet matches any of the rules, the corresponding action will be taken, either raising an alert (A), dropping the packet (D), or switching states in the internal state machine linked to the rule (E<state_number>).

NOTE: If a packet matches multiple rules, the first in the list to match will be the one considered.

Two instances of this TC Analyzer are used, both with different rule sets, with packets incoming from the Probe 2 and Probe 3 respectively. The reason behind this choice is that some attacks can only be monitored via the Probe 3's Software Bus location, because packets internal to the satellite are needed for detection. However, as we want to detect / prevent attacks as soon as possible, for attacks coming from the ground, the Probe 2's input is the best option.

To edit the rules and re-compiled the related .c files, refer to the **IDS README**.

NOTE: The TC Analyzer only takes input starting from the Probe 2 as it needs to be able to access the deciphered contents of a packet.

Another important component of the detection is the Task Profiling module, located in the *ids_p3.c* file. It consists in a mapping of all allowed messages for each application Task present in the satellite. For each Task name, all allowed commands are listed. If a packet violates the allowed command set for a given Task, it is blocked.

There is an exception for the "CI Custom Main Task" (packets coming from the ground), for which packets not present only raise a warning, as during satellite manoeuvres or recovery procedures unseen / rare commands can be legitimately sent from the ground and may not be present in the list.

WARNING: New Tasks or code modification of Tasks can lead to legitimate packets being blocked by this module. Make sure to update this list accordingly !

NOTE: This module can be disabled by commenting the P3_PROFILING_ENABLED define in *ids_probes.h.*

For additional information regarding these modules or the other ones not mentioned, you can refer to the paper "Attack Scenarios and Embedded Intrusion Detection for Space Systems" linked in Section 27.

18 Input Generator

The user can generate TCs for the constellation by using COSMOS, but also by using the Input Generator defined in CSS project. The Input Generator is based on python scripts.

This utility can be used both for legitimate mission operation, as well as for attack scenarios.

In order to use the Input Generator, the user shall first create a .json file corresponding to the desired commands (for example) :

```
python3 Main.py create scenarios/test.json
```

then the user should adapt the .json file manually (if necessary) to modify the default parameters of the commands. Finally, the user can send the command using:

```
python3 Main.py send scenarios/test.json
```

Using the "--port" option you can also specify the port.

```
python3 Main.py send scenarios/test.json --port 6012
```

The Input generator should be used on the VM MCS in order to send the commands to CryptoLib input port.

Fuzzing and timestamp fields development is in progress. The feature is not fully available in this version of the simulator.

The user can create combinations of commands directly with the create argument or by using a dedicated script. For example :

```
python3 Init_mission.py 6010
```

Using only the CmdParser, the user can also get the syntax of a command via :

```
python3 CmdParser.py
```

Currently, the commands available are parsed from the COSMOS configuration files located in the */cmd_tlm/<target> folders.

Automatic endianness conversion is supported.

19 Custom countermeasures and specific NOS3 modifications

Specific countermeasures or improvements are proposed by default in CSS simulator:

1. TTL (Time To Leave) information is added by default in front of each TF (1 octet) in order to avoid loops into the constellation network (see Figure 33). A counter about dropped packets is available in ISL Telemetry. In order to deactivate TTL check in, one can use *ISL_TTL_FEATURE=0* in *isl_app.h*.
   Warning: ISL_TTL_INIT in ISL and TTL_INIT in Front-End shall be the same.
2. A check for new routing tables received by TC is added by default in ISL to avoid loops. Comment the *#define ISL_LOOP_CHECK* in *isl_app.h* to deactivate this feature.
3. A dummy MAC is introduced to identify some Critical TCs as *CFE_ES_STOP_APP* and *GENERIC_ADCS_SET_MODE_CC* (end of the SPP packet). One can use the same approach to define other critical TCs. MAC is temporary defined as “1234” only for prototyping.
4. Watchdog messages are exchanged among the satellites via the ISL (as TM messages).
5. Operator reaction to IDS detection of malicious payload is simulated via Cryptolib standalone: The operator reload on board a legit version of the camera payload.

Watchdog messages implementation is distributed between the ISL and Front-End. In particular, the ISL uses TM like messages containing the “*DEADBEEF*” sequence and the Front-End uses TC like messages. If a watchdog is not received for some specific time, a warning message will be shown on the FSW (and Front-End tab).

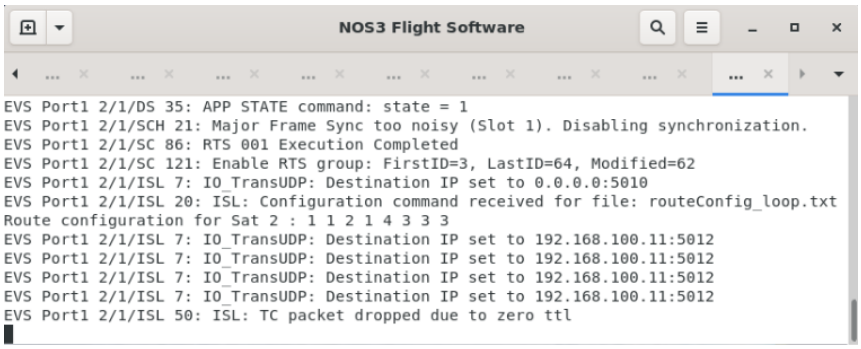


Figure 33. Example of packet dropped due to TTL.

20 CFDP

The CFDP layer inherited from NASA NOS3 v1.6.2 is not fully functional. In particular the uplink command from COSMOS and the COSMOS decoding of CFDP packets from CF application. For this reason, a workaround solution to be able to use CFDP in CSS simulator has been considered. The workaround is based on the following choices:

- add an external python library : https://gitlab.com/librecube/lib/python-cfdp
- add custom CFDP commands for uplink and downlink :CF_UPLINK_FILE (MsgId 0x18B3), CF_DOWNLINK_FILE (MsgId 0x18B6) commands.

The origin filestore is "*home/nos3/Desktop/github-nos3/fsw/build/exe/cpu1/cf*".

In Figure 34 a diagram is presented to clarify the integration of the external library into the NOS3 architecture.

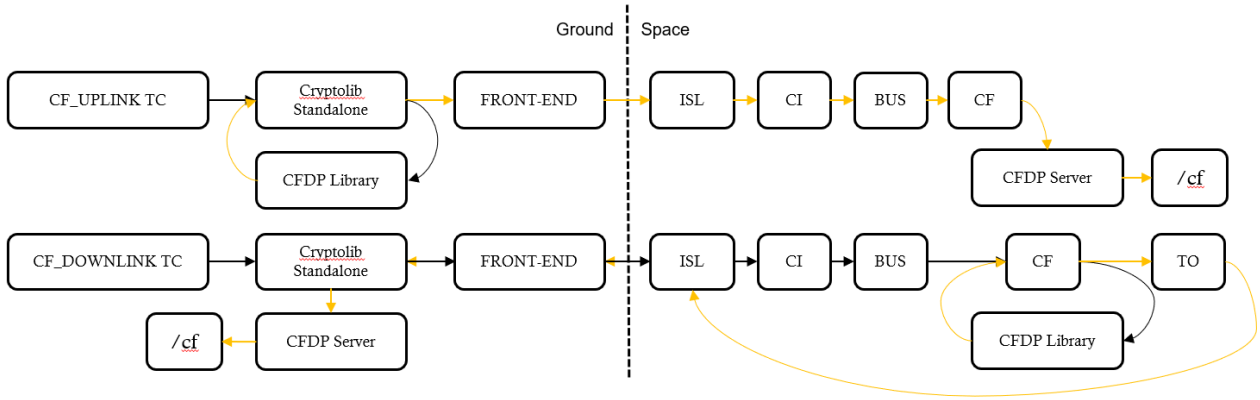


Figure 34. Example of CFDP Class 1 implementation using external library.

In practice, when the simulator start, a CFDP server tab is opened for each satellite and also several CFDP server tabs are opened on ground (one for each satellite in the constellation).

The CFDP server listen for incoming CFDP packets and it generates the file in the filestore (/cf). This approach enables us to exchange files using CFDP and above all to have a realistic CFDP traffic on the space link.

WARNING: some custom modifications have been added to the original library. In particular, concerning the default size of CFDP packets (for compatibility reasons with CryptoLib and cFS TF management).

WARNING: CFDP class 2 is not available in this version of the simulator. It can be added in further releases using the same external library.

21 Large Scale Dataset Generation

One advantage of having a faithful simulator lies in its ability to generate relevant data without the pre-emptive cost of collecting data using a real-world system. In addition, a simulator that can be parameterized and parallelized allow scaling up the quantity of data that one can generate.

Enabling embedded ML-based intrusion detection in satellite systems requires more than isolated simulation runs or ad hoc data collection. Instead, it calls for a structured strategy that transforms the CSS simulator into a dataset-generation enabler, capable of supporting large-scale, reproducible, and systematically annotated experimentation campaigns. Thus, a companion repository css-dataset (https://github.com/IRT-Saint-Exupery/css-dataset) has been developed.

This section presents the design strategy adopted in that companion repository with the design requirements that guide the organization of simulations, data, and orchestration mechanisms. However, the how-to and details will be found in the documentation of the dedicated repository.

The primary objective is to enable the generation of ML-ready datasets for embedded intrusion detection, while preserving the realism of the satellite missions and system interactions already modeled in the CSS-Simulator. In particular, the strategy aims to:

(i) scale to large volumes of data through automated experimentation,
(ii) generate diverse and realistic simulation configurations,
(iii) precisely trace attack execution and derive reliable annotations,
(iv) efficiently post-process raw simulator outputs into ML-friendly formats,
(v) exploit parallel execution capabilities when hardware resources allow it,
(vi) and provide an end-to-end pipeline from simulation specification to dataset publication.

From the objectives above, we derive the following design requirements (DRs):

• **DR1 — Declarative simulation specification**: a single simulation run must be fully defined through explicit configuration files.

• **DR2 — Scalable variability**: the framework must support the generation of large numbers of relevant missions (DR2.1) and simulation configurations (DR2.2).

• **DR3 — Scalable and shareable dataset organization**: generated datasets must be organized to scale to large volumes and be reusable by the community.

• **DR4 — Post-processing to ML-ready formats**: raw simulator outputs must be transformed into compact, structured data formats.

• **DR5 — Automated dataset update and publication**: simulation outputs must be automatically integrated into dataset repositories together with their metadata.

• **DR6 — Parallel experiment orchestration**: multiple simulation runs must be orchestrated across available computational resources.

The repository address all this challenges. Basically, if one has deployed an infrastructure as the one in Figure 35, then the companion repository can help to deployed the workflow

depicted in Figure 36. Such a workflow help to generate a dataset with the architecture illustrated in Figure 37.

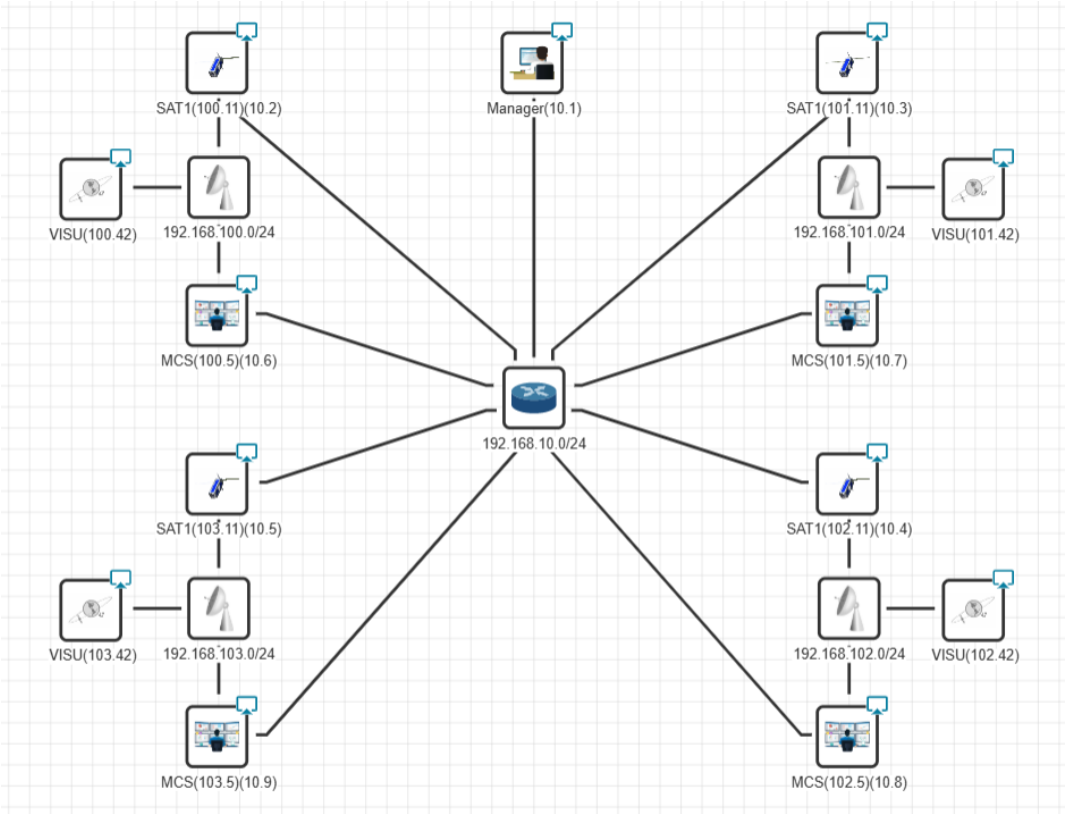


Figure 35. Example of scenario for data generation: 1 SAT x 4.

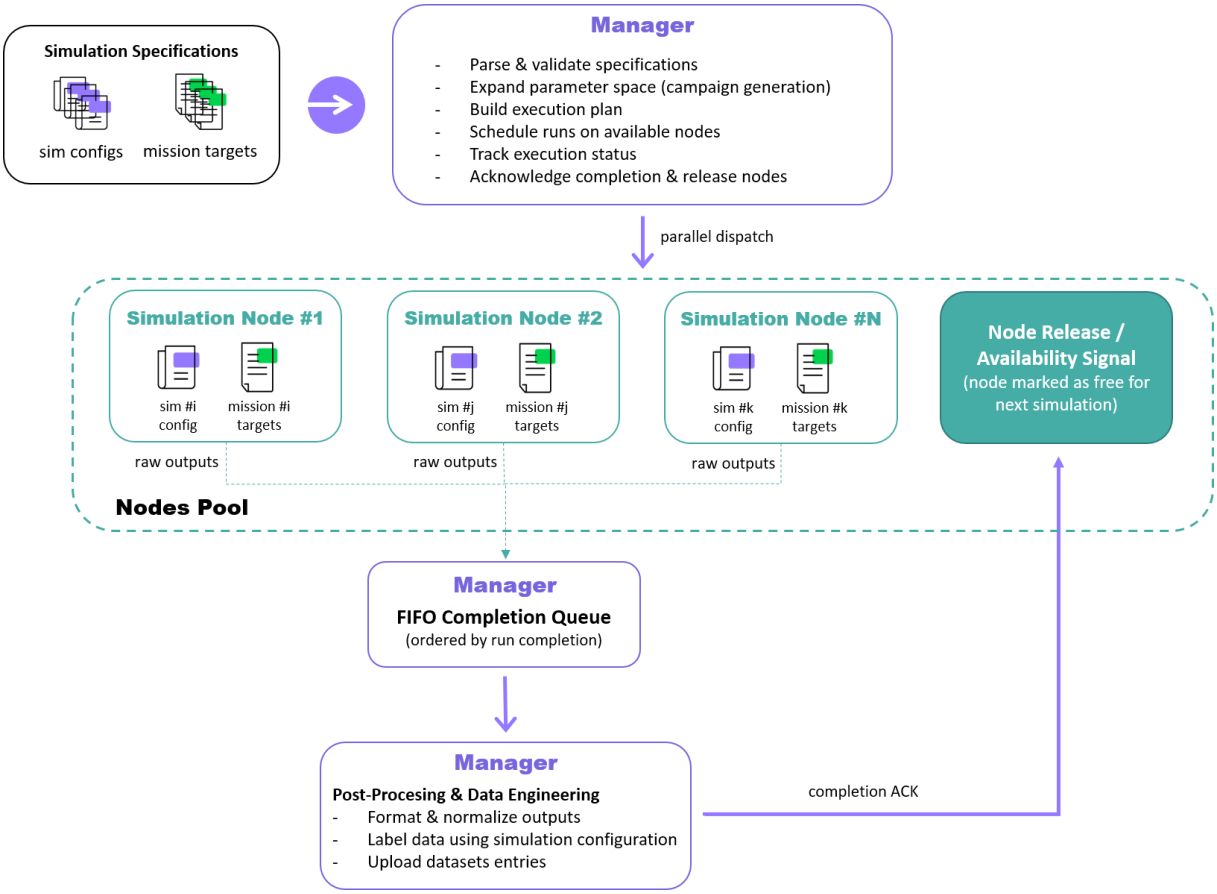


Figure 36: Overview of the simulation orchestration workflow. The manager dispatches simulation runs to a pool of nodes for parallel execution. Completed runs are queued and processed sequentially in a FIFO (First-In, First-Out) post-processing pipeline.

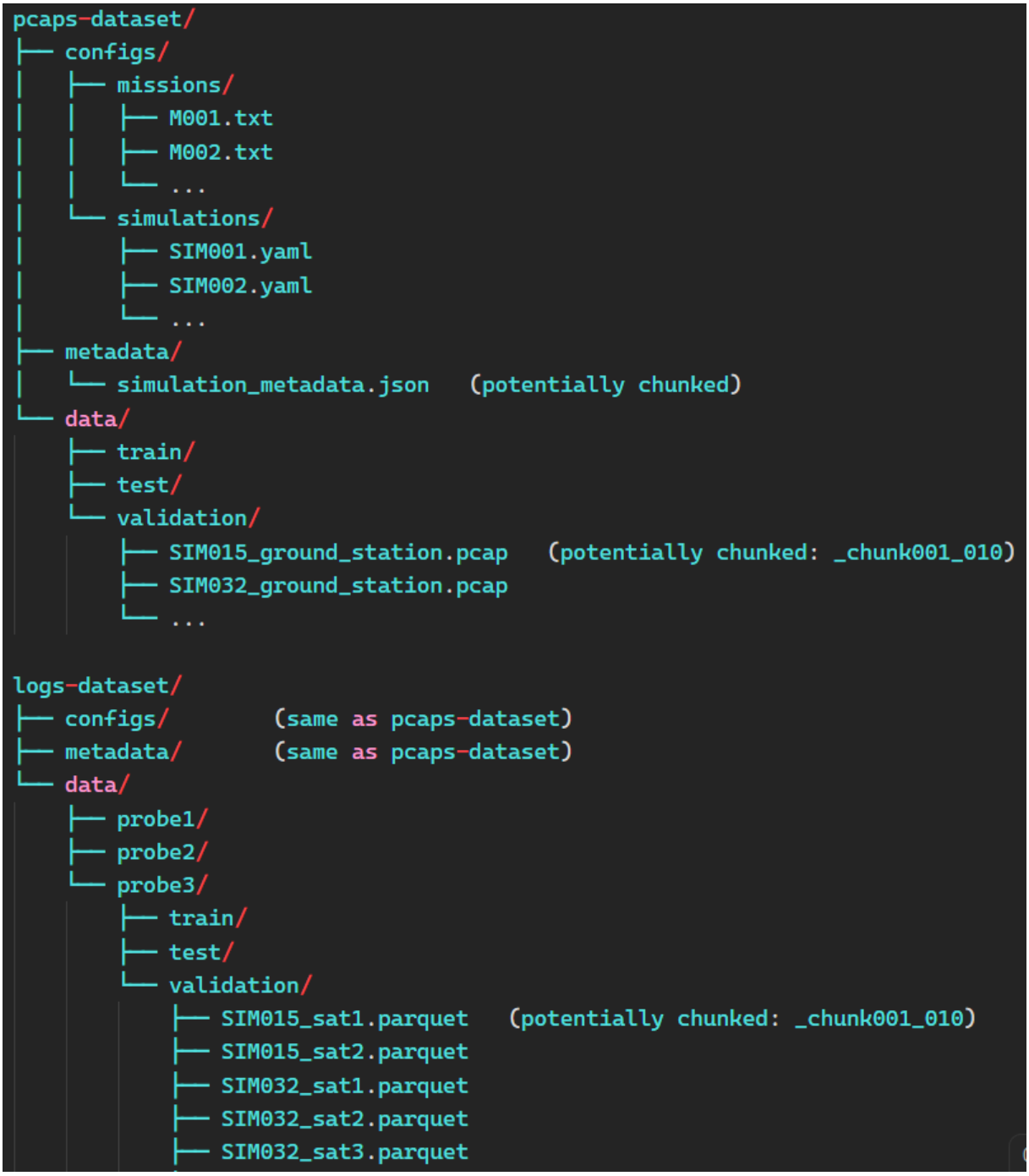


Figure 37. Datasets organization for ground-segment PCAPs and onboard IDS probe logs.

22 Design choices and simulator philosophy including limitations

It is important to discuss here some architecture and design choices of the simulator used for CSS project.

Please note that the CSS Platform is composed by multiple repositories :

1. */home/nos3/Install_tools_nos3* : this is to deploy a constellation and to diffuse updates (see Procedure.txt for all the detailed steps). Example to diffuse updates to all satellites: *./Do_Update_All.sh*

2. */home/nos3/CSS_Attacks* : this is the folder containing the exploits developed for CSS project. For help *./Master_Attacks_Scripts.sh 0*

3. */home/nos3/Input_Generator* : the python code of the input generator is here.

4. */home/nos3/Desktop/github-nos3* : the modified NOS3 code is here (including ISL and IDS/IPS). Example to launch the simulator : *./start.sh* (*./stop.sh* to stop the simulator).

5. */home/nos3/eclipse-workspace/Front-End* : the Front-End component code is here.

6. */home/nos3/eclipse-workspace/imager* : the imager component code is here.

7. /*home/nos3/eclipse-workspace/mission* : the automatic mission component code is here.

8. */home/nos3/Scenario_Manager*: the code of the scenario manager is here.

9. */opt/nos3/* : NASA 42 and COSMOS code is here. Cosmos folders are duplicated for each satellite in the constellation (cosmos1, cosmos2, etc).

10. /home/nos3/css_dataset: this will contain the companion repository described in the **Large Scale Dataset Generation** section. It will provide tools for automatic simulation run and data generation. See https://github.com/IRT-Saint-Exupery/css-dataset . **N.B:** It is possible that at the time of publication of this document the repository is not yet available. If you want to know the progress please reach out to us.

WARNING:

- The simulator architecture is **adapted only for small constellations**.
- **The RF layer is not simulated**.
- The following functions are not available or not working or not updated in the current release:
  - **COP-1 layer is not available (only partial implementation)**
  - **encrypted TLM is not working properly (not available by default)**
  - **ADCS and other components telemetry is inactive by default to limit traffic (TLM should be activated in Scheduler and TO tables)**
  - **Cryptolib is not updated lo last available version to keep existing vulnerabilities.**

The simulator is distributed in the same fashion as NOS3 (as a single VM), but is has been decided to use multiple VMs to run the simulator. This choice is to increase modularity and to

have different operators on different VMs (for demo and trainings). However, we consider a single VM approach for the development (**we develop only on the main VM and we run on multiple VMs**).

The simulator is based on a fixed version of NOS3 v1.6.2. This choice is related to available resources and the additional burden required to follow NOS3 development.

We have introduced new components and functions but the simulator is still missing some important elements (ex. COP-1 layer). Again, this is related to available resources and it is the result of a trade-off. Please do no hesitate to contribute and to improve the simulator for the community.

23 Deployment procedure

The CSS Simulator is based on multiple VMs: 1 VM for the Mission Control System (COSMOS), 1 VM for Flight Dynamics and Visualization (NASA 42), 1 VM for each satellite in the constellation (NASA cFS).

The "comfortable" resources CPU/RAM(GB) required to run the simulator can be summarized as follow :

1. **Satellite VM** : 2CPU / 3GB or higher.

2. **Space Environment VM** : 4CPU / 4GB or higher.

3. **MCS VM:** 1CPU / 4GB for each satellite to be controlled (or higher). **NB**: if only 1 satellite is deployed, then 3 CPU / 5.5GB is recommended for MCS. COSMOS (MCS) is quite sensitive to RAM allocated, and if under-provisioned, services start can timeout indefinitely.

The VMs should share the same network (for deployment). The VMs are all defined from a generic VM (based on Ubuntu 20.04 LTS, same as NOS3 VM) that can be specialized after creation.

The preferred method to deploy CSS simulator is via the main generic VM of the project. The link to download the main VM is provided in the **README file of CSS GitHub repository**.

The CSS simulator has been defined and deployed in a proprietary cyber range environment (**CITEF** by Nexova). However, it is possible to deploy and test the simulator locally or on dedicated hardware using for example VirtualBox (as for NOS3).

For deployment and installation, once you have downloaded the main VM, there are two options.

1. You have access to CITEF (or similar software) and you have created your scenario and the associated network. In this case you can simply log into the VM representing the first satellite of your constellation (SAT1) and you can follow the steps described in the *Procedure.txt* file in */home/nos3/Install_tools_nos3* (see Deployment Procedure subsection below).

2. You have access to Virtual Box (or similar software) then you have to create your scenario and the associated network manually. To do so you can follow the steps in Figure 38. The generic VM is “multiform” and has to be replicated and then specialized in order to become either 42, MCS or sat<i> (see Deployment example for VirtualBox subsection below).

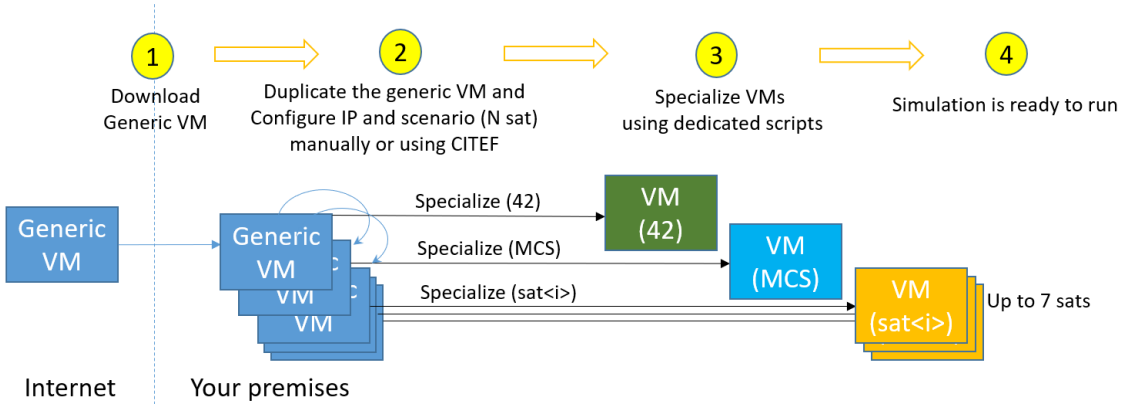
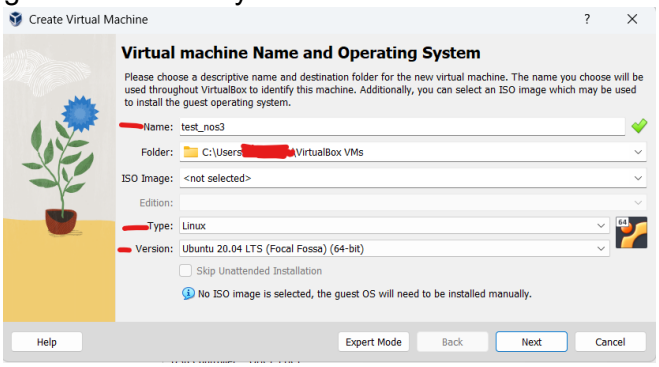


Figure 38. Example of deployment procedure.

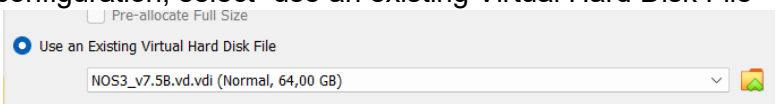
WARNING: The deployment is currently limited to 7 satellites but this limit can be modified with some effort (see Tips & Tricks section).

23.1 Deployment example for VirtualBox

1. Base VM configuration:
    a. Decompress the VM disk file into a folder of your choice
    b. In VirtualBox, click “New” button at the top centre of the screen
    c. Fill-in the name of VM with the desired name, with type of OS and Version as the following, leaving all other fields by default:



    d. Fill the hardware requirements as described in the previous page
    e. For the disk configuration, select “use an existing Virtual Hard Disk File”

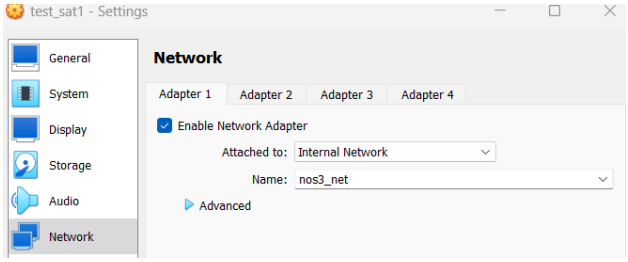


    f. Click on the folder icon on the right, and add the decompressed folder of the VM as the VM disk with the “Add” button.

2. **Optional:** If you want the latest updates, start the base VM with the network settings set to NAT mode (to give access to internet to the VM, usually default) and **pull the latest updates from GitHub** in the home folder.

3. Clone the VM with Right Click -> Clone, and change the name to the desired name. Be careful to leave the MAC address settings “as is” and only include NAT network adapter addresses. **Select the “full clone” mode**.

4. For every VM, edit the network configuration via Settings -> Network and select the “Internal Network” mode with the desired name (must be the same for all VMs). This will create a virtual network between all VMs:



5. Start the VMs and edit the */etc/network/interfaces* file to change the **static IP** of the VM (this allows to keep this IP even after reboots). You can also use Network Manager or other utilities to handle IPs.

6. The VMs are initialized, and you can follow the steps described hereafter.

23.2 Deployment procedure

NB: it is suggested to pull the latest modifications from GitHub before performing the deployment procedure. In particular, the repository is in the Home directory of the VM, moreover, you can also pull the latest updated from the css-dataset directory (separate repository).

The deployment procedure requires to run a few scripts on VM SAT1 and VM MCS in order to specialize and prepare the simulator:

1) VM SAT1: Modify NBSAT (and IP if necessary) in *IP_Scenarios.sh* (See Figure 40).
2) VM SAT1: call *Simple_Deploy.sh* and wait the script to complete (See Figure 41).
3) VM MCS: call *launch_containers.sh* (see Figure 42).
4) VM MCS: call *make* (see Figure 43).

NB: While the make command completes quickly, COSMOS can take a while to start correctly (2-10mn, depending on resources) You can check if the interface is connected and if the launch is complete by connecting to the “localhost:2900” webpage in Firefox and looking for new log messages that may indicate startup. If COSMOS is stack in start loop for more than 15min it is suggested to restart and allocate more RAM. You may be prompted to setup a password, we recommend “nos3123!”, as the VM password. Look for the following:

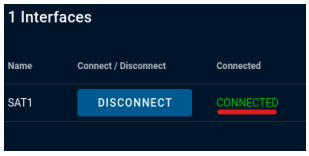


Figure 39. Cosmos interface connected

Once all the script are completed you can launch the simulator using *./start.sh* (see Figure 44).

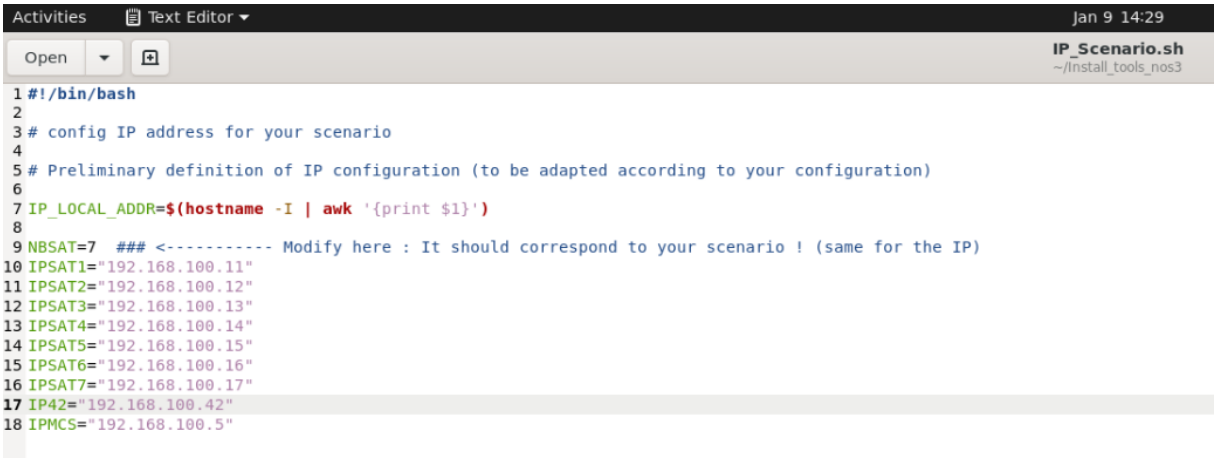


Figure 40. Verify that NBSAT and IP address are correct in IP_Scenario.sh in VM SAT1.

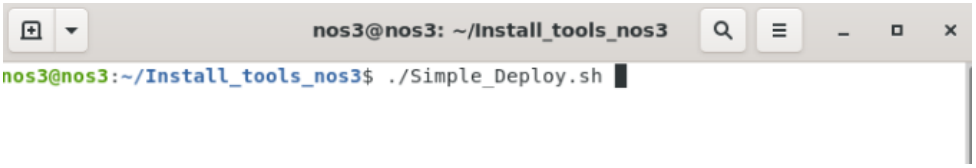


Figure 41. First step of deployment from VM SAT1.

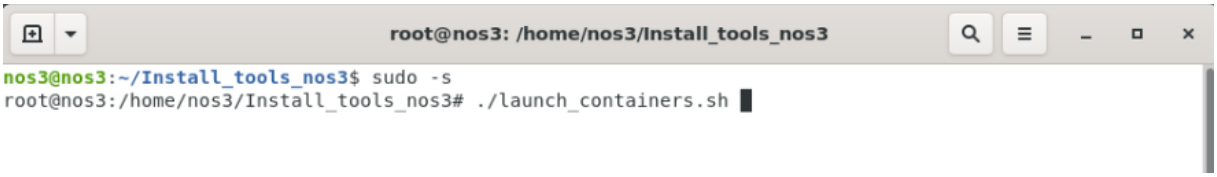


Figure 42. From VM MCS you need to launch all COSMOS Docker containers.

Figure 43. From VM MCS you need to compile COSMOS interfaces.

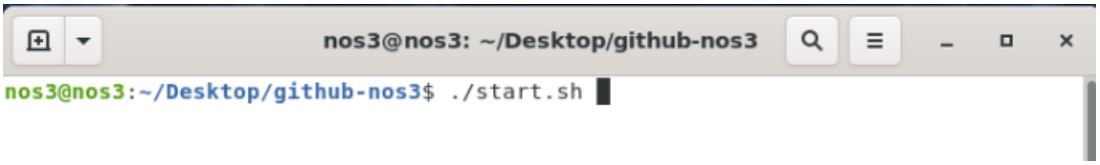


Figure 44. Once all deployment step are completed you can call START from VM SAT1.

NB: The deployment procedure can be fully automated using Ansible. An example for 2 SAT is provided in folder *Install_tools_nos3/ansible_deploy*.

NB: for CSS project we have deployed a 7 satellite constellation including proprietary ground probes as in Figure 45.

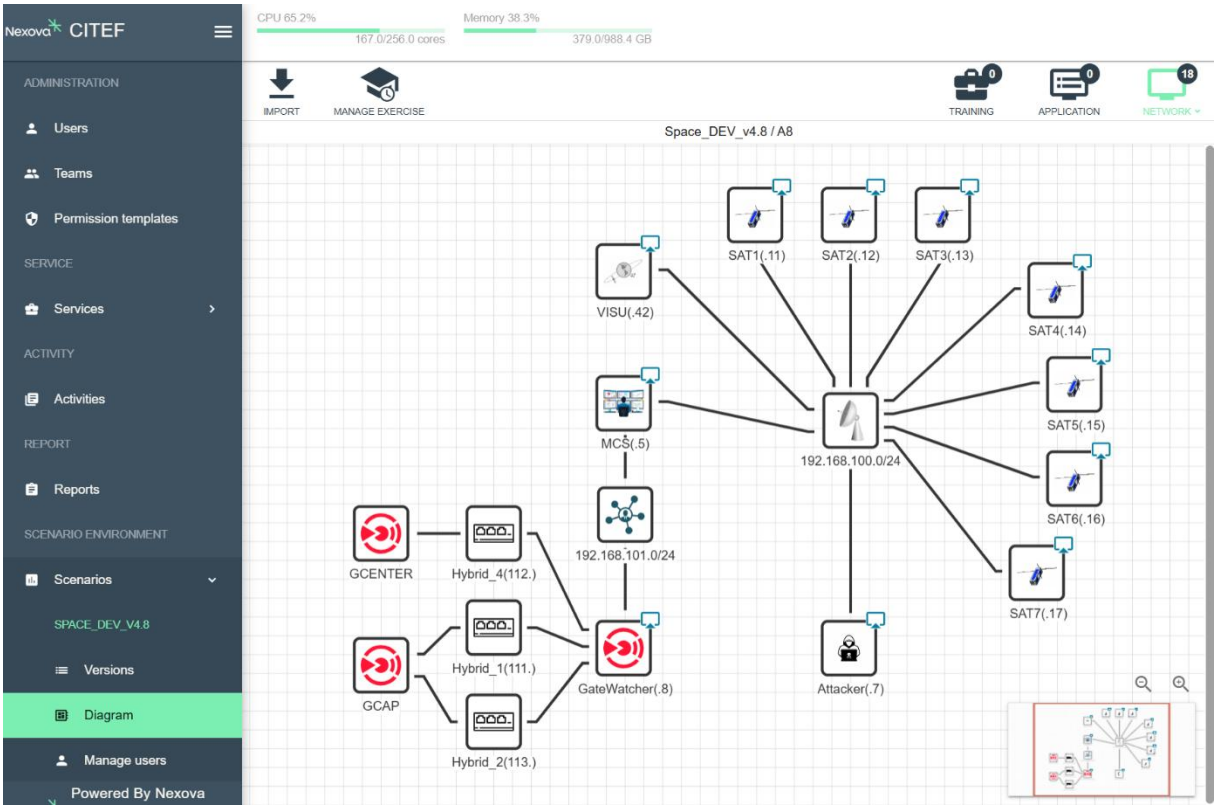


Figure 45. Example of CITEF Scenario including the space system simulator.

24 Usage example

To launch the simulator just log to the first satellite (ex. SAT1(.11)) and use **./start.sh** (./stop.sh to stop the simulator) in **github-nos3 folder.**

After launch (generally 1 to 2 minutes) :

1. NOS3 flight software tabs will be available in each SAT VM.
2. COSMOS GUI for TC/TM will be available in MCS VM. A web tab will be available for each SAT in Firefox. Moreover, the operator will have access to CryptoLib Standalone tabs and Front-End tabs.
3. CFDP servers tabs will be available in MCS VM and in each SAT VM.
4. Satellite/Constellation visualization and space environment simulation (NASA 42) will be available in VM VISU. The considered ground stations are visible in the 2D map.
5. The Camera Imager (what the Arducam can see) will be available in each SAT (**NB**: Imager is updated only if SAT is in "Normal Mode").

The automatic mission defined in mission component will start a few minutes after starting the simulator. More details are presented in the dedicated section.

Another possibility to launch the simulator and gather relevant data is via the Scenario manager component or via the css-dataset repository. More details are provided in the dedicated section.

A live demonstration of CSS simulation platform has been presented at:

- CYSAT 2025 : https://cysat.eu/cysat-europe
- Starion/Nexova Tech Talks: https://youtu.be/7SkSOk2pjhM?feature=shared

For information, useful simulation data are temporary stored in /tmp folder.

- in /tmp folder of MCS VM
  - decoded photos during the mission are available
  - visibility and position (ECEF) information for all satellites
  - satellites occupation (busy taking photo or not)
- in /tmp of SAT VM you can find
  - the input file for imager
  - the logs generated by IDS for detection
  - control errors for ADCS normal mode
  - visibility and position (ECEF) information

25 Development example

The user can customize the different components of the simulator by working on the main VM of the simulator (SAT1).

We suggest to modify only SAT1 and then to diffuse the modifications. This is managed via bash scripts in *Install_tools_nos3 (see* Figure 46*).*

To diffuse updates to all VMs :

```
./Do_Update_All.sh.
```

After update, you should always compile:

```
./Do_Make_All_VM.sh.
```

However, only for modifications in COSMOS telemetry or telecommands files, the user should compile (as root) also on MCS VM.

You can clean all build using

```
./Do_MakeClean_All_VM.sh.
```

Front-End, Imager and Mission have been developed using **eclipse** (pre-installed in the VM). The user can use eclipse to modify and compile these components.

Cryptolib standalone is compiled at launch on MCS VM. For CryptoLib modifications check at launch on MCS VM.

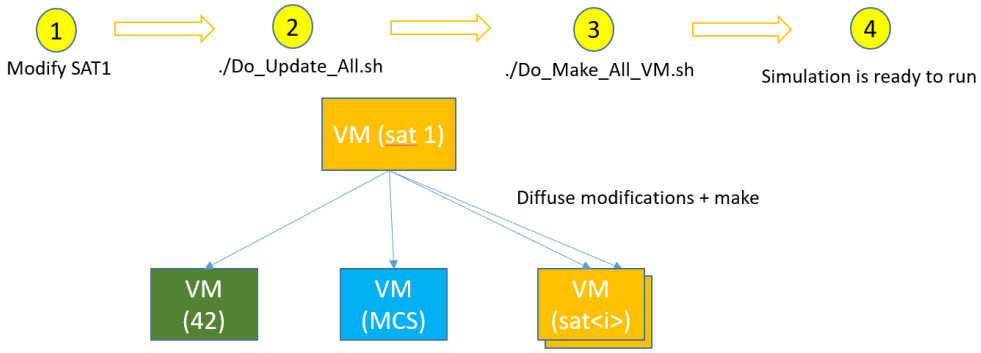


Figure 46. Suggested development approach.

26 Tips & Tricks

TIME:

The simulation time is not real time. The simulation is slower than real time by default. You can check Sim Time in 42 window to estimate the ratio.

In order to change step time you have to modify the following files :

- *Inp_Sim.txt* (0.01s by default)
- *SC_NOS3.txt* (sample time for all components shall be bigger or equal to Inp_Sim.txt)
- *nos3_simulator.xml* (10000 by default)
- *cfe_psp_start.c* (100 to have 0.01s)

VM SIZE:

The VM size can increase during the simulator use, the user can clean old files by regularly doing a ./Do_MakeClean_All_VM.sh + ./Do_Make_All_VM.sh.

LAUNCH:

The launching scripts are defined in the folder :

/home/nos3/Desktop/github-nos3/gsw/scripts

You can adapt the components to be launched and other start settings in these scripts.

In the same folder the user can find several utilities.

MORE SATELLITES:

The simulator is defined to handle by default from 1 to 7 satellites, but this number can be increased (if the hardware capacity is available).

The main modifications to be considered are :

- add new cosmos instances for each satellite (/opt/nos3/cosmosX), see example in /Install_tools_nos3/misc
- add new CryptoLib standalone instances (/Desktop/github-nos3/components/cryptolib/support/standalone)
- add new NASA 42 sockets and adaptations (/install_tools_nos3/Specialize_42_InOut)
- update Install_tools_nos3 scripts
- update automatic mission

It is important to note that another limiting factor is NASA 42 sockets availability/stability. The more the number of satellites, the higher the number of sockets, the lower the stability of the sockets and the simulation. A different approach in NASA 42 sockets architecture should be considered for bigger constellations.

ACTIVATE/DEACTIVATE ENCRYPTION:

The encryption on TC can be activated by typing VCID 4 in CryptoLib Standalone tabs on VM MCS, but also by using the dedicated script in */gsw/scripts.*

FASTER / SLOWER:

There is an embedded function in NOS3 (NOS Time Driver tab) to accelerate or decelerate the simulation. A script is also provided in */gsw/script* to call the NOS3 Time Driver tab.

ADD NEW CFS COMPONENT:

In order to add a new cFS component, as for example thrusters, the user can have a look at :

Generating a New Component — NOS3 documentation

IF I NEED A TELECOM PAYLOAD ?

In order to add a telecom payload, the user has not only to add a new component, but also to adapt the routing components. Indeed in a telecom constellation satellites often have up to 4 neighbours allowing to create alternate tables in case of failure (or congestion).
Current routing in ISL has to be enhanced to manage alternate routes.

Another important difference is that in an observation mission, traffic is generated from inside the constellation via taking pictures.

In CSS we have encapsulated it in TM flow, noting the source of a packet as the id of the satellite that produced this packet. This is for TM. The packet destination for TM is always set to “ground”. In a telecom mission in addition to TC and TM, communications come from the Earth and are destined to the Earth. So, update would be necessary to manage this new kind of packets.

IF I NEED TO UPDATE THE NOS3 VERSION ?

CSS Simulator has been developed starting from NOS3 v1.6.2. Indeed, it has been decided at the beginning of the project to build on top of this version without following the official NOS3 developments. This choice is related to available resources and the additional burden required to follow NOS3 development. The scope and architecture of CSS simulator is quite different from NOS3 and a direct reintegration of a recent version of NOS3 is not directly possible without breaking something. Moreover, NOS3 is evolving trough a full “dockerization” of the simulator. Nevertheless, for the majority of cFS components, CryptoLib and other features, the user can manually get the new projects updates (if necessary/useful) from the NOS3 repository GitHub - nasa/nos3: NASA Operational Simulator for Small Satellites.

NOS3 and its documentation have largely evolved from NOS3 v1.6.2. You can find more details here:

Home — NOS3 documentation

IF I NEED TO CHANGE THE ABSOLUTE START TIME ?

In order to modify the start time of the simulation the user has to modify at least the following files : *Inp_Sim.txt* for NASA 42 and *nos3-simulator.xml* for NOS engine. We have created a dedicated script in */gsw/scripts* called *diffuse_nos3_time.sh* that automatically updates all necessary configuration files across your network.

Simply call:

```
./diffuse_nos3_time.sh <delta_seconds>
```

Where <delta_seconds> is the time offset in seconds from the default simulation epoch (T0 = 2025-10-18 08:30:00 UTC). The script will:

- Update *nos3-simulator.xml* and its CMake build copy
- Update *Inp_Sim.txt* with the corresponding date/time
- Propagate changes to all VMs (42, MCS, SAT1-SAT7)

Then run *./start.sh* as usual, and the simulation will start at T0 + delta seconds.

NB: T0 = 18/10/2025 8h30 in *nos3-simulator.xml*. T0 is indicated in seconds wrt to J2000.

IF I NEED TO CHANGE THE ORBIT ?

The user can change the orbital parameters defined in the files (example for SAT1) :

/sims/cfg/InOut/Orb_ISS_sat1.txt and
Install_tools_nos3/Specialize_42_InOut/Inp_Sim.txt.1SAT

The default parameters are the following :

10 18 2025 ! Date (UTC) (Month, Day, Year)

08 30 00.00 ! Time (UTC) (Hr,Min,Sec)

37.0 ! Leap Seconds (sec)

1300.0 1300.0 ! Periapsis & Apoapsis Altitude, km Apoapsis Altitude (km)

1300.0 0.0 ! Min Altitude (km), Eccentricity

100.822 ! Inclination (deg)

22.0 ! Right Ascension of Ascending Node (deg)

202.7876 ! Argument of Periapsis (deg)

257.145 ! True Anomaly (deg)

The user can change orbital parameters using the dedicated script *diffuse_nos3_anomalies.sh* to update True Anomaly across all VMs:

```
./diffuse_nos3_anomalies.sh SAT_ID:TRUE_ANOMALY_DEG [SAT_ID:TRUE_ANOMALY_DEG
...]
```

Example for a 7-satellite constellation:

```
./diffuse_nos3_anomalies.sh 1:257.145 2:308.574 3:0 4:51.429 5:102.858
6:154.287 7:205.716
```

This script automatically updates *Orb_ISS_sat<N>.txt* on all VMs with the correct anomaly values.

For **other orbital parameters** (altitude, inclination, RAAN, argument of periapsis), manually edit:

- */sims/cfg/InOut/Orb_ISS_sat<N>.txt* (on each SAT VM)
- */Install_tools_nos3/Specialize_42_InOut/Inp_Sim.txt.<NBSAT>* (source template)

Then propagate via *Do_Update_All.sh* + *Do_Make_All_VM.sh*.

NB: The input files used by NASA 42 are in */opt/nos3/42/InOut* folder of VISU VM, but we suggest to modify the files in SAT1 VM (see *Install_tools_nos3/Specialize_42_InOut*) and to diffuse the modifications via the procedure presented in this manual : *Do_Update_All.sh* + *Do_Make_All_VM.sh.*

IF I NEED TO USE DIFFERENT IP FROM THE DEFAULT ONE ?

By default the scenario is based on the following list of IP (see IP_Scenario.sh):

IPSAT1="192.168.100.11"

IPSAT2="192.168.100.12"

IPSAT3="192.168.100.13"

IPSAT4="192.168.100.14"

IPSAT5="192.168.100.15"

IPSAT6="192.168.100.16"

IPSAT7="192.168.100.17"

IP42="192.168.100.42"

IPMCS="192.168.100.5"

We suggest to keep this type of address to avoid issues.

However, the user can decide to define different IP, for example of the type 192.168.101.xx.

In this case, it is important to observe that (in this version of the simulator) IP are hardcoded in several files and these files have to be updated for the simulator to work properly. Moreover, the deployment procedure based on bash script requires the ssh keys to be generated in advance to avoid errors.

Finally, for deployment with alternative IP address, we suggest to :

1) generate manually all the necessary ssh keys in advance for the new IP (= simply test ssh connection from all SATX VM to the other VMs, ex. *ssh nos3@192.168.101.11* , etc).

2) update the IP in *IP_Scenario.sh.*

3) launch the *Simple_Deploy.sh* script.

4) deploy Cosmos in VM MCS (as explained in *Procedure.txt*)

NB: Be careful to update manually the variable *SAT_IP_ADR_FMT "192.168.100.%d"* in ISL component.

NB: Be careful with ISL function *'ISL_IpFromSatId'* as it is not fully adapted to generic IP in this version. Modify the function if necessary.

IF I NEED TO CHANGE NOS3 TABLES ?

The user may need to adapt the scheduler tasks for many reasons. For example to add new regular tasks for the FSW or to increase/reduce the frequency of a specific call.

The user should modify the files :

- *sch_def_msgtbl.c* (task number and definition)
- *sch_def_schtbl.c* (task list)

For more details about regular tasks and scheduler architecture please have a look at Home — NOS3 documentation.

IF I NEED TO MODIFY THE IDS ?

To edit the Probes / data collection edit the following files:

- Probe 1 : *components/isl/fsw/src/isl_app.c*
- Probe 2 : *fsw/apps/ci/fsw/examples/udp_tf/ci_custom.c*
- Probe 3 : *fsw/cfe/modules/sb/fsw/src/cfe_sb_api.c* (with init in *cfe_sb_init.c*)

To edit the IDS logic / add new modules, you will need to edit / add new files in the *components/ids/fsw/src* folder.

WARNING: When adding new files update the CMakeLists.txt of the component calling functions in your file accordingly, else compilation will fail or runtime errors can appear ! (at start of sim)

IF I NEED TO GENERATE A DATASET USING THE SIMULATOR ?

We have developed a repository companion dedicated to this use. The principles are briefly described in the section **Large Scale Dataset Generation** but you can also directly check the Github repository (https://github.com/IRT-Saint-Exupery/css-dataset).

N.B: It is possible that at the time of publication of this document the repository is not yet available. If you want to know the progress please reach out to us.

IF I NEED TO KNOW IF A SATELLITE IS BUSY TAKING A PICTURE ?

The user can verify the */tmp/isbusy.txt* file in VM MCS. The file contains a 0 (free) or 1 (busy) for each satellite in the considered constellation.

27 Publications (04/2026)

- **ESA 3S 2025** (A Satellite Constellation Simulator for Space Systems Cybersecurity Research and Development | Security for Space Systems (3S) conference): A satellite constellation simulator for space systems cybersecurity research and development. (*Introduction to CSS platform*)
- **IAC 2025** (A Novel Simulation Platform for Space Systems Cybersecurity R&D — IAF Digital Library): A novel simulation platform for space systems cybersecurity R&D. (*Focused on attacks and mitigations*)
- **EDCC 2026** (https://www.cs.kent.ac.uk/EDCC2026/home): Attack Scenarios and Embedded Intrusion Detection for Space Systems.

If you find this work useful, please acknowledge it by citing these papers.

28 Disclaimer

NOS3 is distributed under the NOSA 1.3 License that can be found in github-nos3 folder.

The CSS simulator is also distributed under the NOSA 1.3 License. See `LICENSE.txt` for more information.

The provided content is for research and testing. We are not responsible for any inappropriate usage of this content. The code and the main VM are provided as-is for general use. We do not offer dedicated support or troubleshooting assistance.

IF I WANT TO CONTRIBUTE?

If you find bugs/errors, potential improvements or if you simply want to build a better CSS simulator and share it with the community please contact :

benjamin.deporte@irt-saintexupery.com

KNOWN ISSUES AND BUGS

The user may observe errors at simulator start, in particular in NOS3 FSW tab. For example of the type :

- *Request Device data reported error -1*

  It is suggested to restart the simulator if this issue is observed.

- *Cannot shmget Reset Area Shared memory Segment: Invalid argument [...], The cFE could not start*

  This error is indicative of a high usage of shared memory segments. The NOS3 simulator does not always clean those up properly on exit and tends to use new ones often too. Multiple consecutive start/stop cycles of simulation can cause this issue.

  The number of memory segments can be monitored with the *ipcs –m* command. The IDs where *nattch* is equal to zero can be removed with the *ipcrm –m <id>* command, but be aware that deleting a memory segment used by another application than nos3 could cause instability/crashes.

  Removing unused segments with the earlier command can be useful if you want to keep working, but when possible you should reboot the VM.

Other types of errors/warnings that can be seen in FSW tab are :

- *GSP input data temporary KO*
- *Star Tracker input data temporary KO*
- *Pipe Overflow, MsgId …, pipe TO_TLM_PIPE_0, …*

If the errors/warnings are only temporary the simulation is not impacted. However, if the errors/warnings persist, it is suggested to restart the simulator.

These types of issues have not been resolved in this version of the simulator, and there are likely additional errors that we are not yet aware of.

To report other issues, please contact: benjamin.deporte@irt-saintexupery.com

29Annexes

29.1 Front-End Sources map

Figure 47. Front-End Sources map.

29.2 Detailed procedure for multi COSMOS deployment :

1. Duplicate the *create_cosmos_gem.sh* file : sat1, sat2, ...one for each sat)
2. Duplicate the */opt/nos3/cosmos* folder (cosmos1, cosmos2, ...one for each sat)
3. Adapt the ports and the associated folders in create_cosmos_gem_satX.sh files

Example of *create_cosmos_gem_satX.sh*:

```
SCRIPT_DIR=$(cd `dirname $0` && pwd)
BASE_DIR=$(cd `dirname $SCRIPT_DIR`/.. && pwd)
GSW_BIN=$BASE_DIR/gsw/cosmos/build/openc3-cosmos-nos3-sat2
DATE=$(date "+%Y%m%d%H%M")

# Start by changing to a known location
cd $SCRIPT_DIR/../cosmos

# Delete any previous run info
rm -rf build

# Start generating the plugin
mkdir build
cd build
/opt/nos3/cosmos2/openc3.sh cliroot generate plugin nos3-sat2

# Copy targets
mkdir openc3-cosmos-nos3-sat2/targets
cd openc3-cosmos-nos3-sat2/targets
```

```
done
echo "" >> plugin.txt
#echo "INTERFACE DEBUG udp_interface.rb 192.168.100.8 5012 5013 nil nil 128 10.0 nil" >> plugin.txt
echo "INTERFACE SAT2 udp_interface.rb host.docker.internal 6012 6013 nil nil 128 10.0 nil" >> plugin.txt
for i in $targets
do
```

4. Adapt networks name and ports to be used in */opt/nos3/cosmosX* via the Ansible file *compose.yaml*

```
networks:
  default:
    name: openc3-cosmos-network-sat2


openc3-operator:
  # For rootless podman - Uncomment this user
  # user: 0:0
  user: "${OPENC3_USER_ID}:${OPENC3_GROUP_ID}"
  image: "${OPENC3_REGISTRY}/openc3inc/openc3
  restart: "unless-stopped"
  ports:
      # - "0.0.0.0:5111:5111/udp"
      # - "0.0.0.0:5013:5013/udp"
      #- "0.0.0.0:12000:12000/udp"
      #- "0.0.0.0:12001:12001/udp"
    - "0.0.0.0:6013:6013/udp"
      # - "0.0.0.0:5011:5011/udp"
```

```
      # - "./openc3-traefik/
    ports:
      - "0.0.0.0:2901:80"
      - "0.0.0.0:2944:443"
```

5. Comment out the ports used in Ansible file compose-dev.yaml (ports of internal containers should not be used)

6. Adapt the *openc3.sh* file :

```
    . "$(dirname -- "$0")/.env"
    args=`echo $@ | { read _ args; echo $args; }`
    (docker network create openc3-cosmos-network || true) &> /dev/null
    docker run -it --rm --env-file "$(dirname -- "$0")/.env" --user=root --network openc3-cosmos-network-sat2 -v `pwd
bin/openc3cli $args
    set +a
```

7. Relaunch docker and COSMOS

8. Adapt NOS3 Makefile and launch.sh file in order to launch all the cosmos containers.

29.3 Detailed procedure for NASA 42 multi spacecraft deployment :

1. Specify multiple satellites in the 42 input files (Inp_Sim.txt)

```
************************  Reference Orbits  ************************
2                                ! Number of Reference Orbits
TRUE   Orb_ISS_sat1.txt          ! Input file name for Orb sat1
TRUE   Orb_ISS_sat2.txt          ! Input file name for Orb sat2
*****************************  Spacecraft  *****************************
2                                ! Number of Spacecraft
TRUE  0 SC_NOS3_sat1.txt         ! Existence, RefOrb, Input file for SC 1
TRUE  1 SC_NOS3_sat2.txt         ! Existence, RefOrb, Input file for SC 2
***************************** Environment  *****************************
```

2. Specify multiple "prefixes" (e.g. SC[0], SC[1], ...) in the 42 input file Inp_IPC.txt to send data over multiple sockets.

```
TX                               ! IPC Mode (OFF,TX,RX,TXRX,ACS,WRITEFILE,READFILE)
0                                ! AC.ID for ACS mode
"IMU.42"                         ! File name for WRITE or READ
SERVER                           ! Socket Role (SERVER,CLIENT,GMSEC_CLIENT)
192.168.100.42     4280          ! Server Host Name, Port
FALSE                            ! Allow Blocking (i.e. wait on RX)
FALSE                            ! Echo to stdout
1                                ! Number of TX prefixes
"SC[0]"                          ! Prefix 1
*********************************  RW 0 to 42   ***************************
RX                               ! IPC Mode (OFF,TX,RX,TXRX,ACS,WRITEFILE,READFILE)
0                                ! AC.ID for ACS mode
"State01_sat2.42"                ! File name for WRITE or READ
SERVER                           ! Socket Role (SERVER,CLIENT,GMSEC_CLIENT)
192.168.100.42     4278          ! Server Host Name, Port
FALSE                            ! Allow Blocking (i.e. wait on RX)
FALSE                            ! Echo to stdout
1                                ! Number of TX prefixes
"SC[1]"                          ! Prefix 1
```

3. Write your sim to parse the data for a specific "prefix", Ex. Sims/cfg/nos3-simulator.xml :

```
<max-connection-attempts>5</max-connection-attemp
<retry-wait-seconds>5</retry-wait-seconds>
<spacecraft>0</spacecraft>
```

NB: be careful to static definition in components folder . Ex. *Generic_rw_hardware_model.cpp* :

```
    sim_logger->error("Invalid torque command value");
}
std::stringstream ss;
ss << "SC[0].AC.Whl[" << _wheel_number << "].Tcmd = ";
ss << torque;

dynamic cast<GenericRWData42SocketProvider*>( sdp)->send comman
```

4. Note that 42 does not pass the NORAD S/C ID or anything like that, so the numbering of the spacecraft is just an index based on the ordering in the 42 config file. Thus, you need to make sure that you match up the right spacecraft number in 42 with the spacecraft number you want in the sim.

5. Modify Tx/Rx folder in */opt/nos3/42* (VM with IP .42)

Example for TX (Inp_IPC.txt)

```
<<<<<<<<<<<<<< 42: InterProcess Comm Configuration File >>>>>>>>>>>>>>
2                                   ! Number of Sockets
**********************************  IPC 1   ****************************
TX                                  ! IPC Mode (OFF,TX,RX,TXRX,ACS,WRITEFILE,READFILE)
0                                   ! AC.ID for ACS mode
"State00.42"                        ! File name for WRITE or READ
SERVER                              ! Socket Role (SERVER,CLIENT,GMSEC_CLIENT)
192.168.100.11     10001            ! Server Host Name, Port
TRUE                                ! Allow Blocking (i.e. wait on RX)
TRUE                                ! Echo to stdout
3                                   ! Number of TX prefixes
"SC[0]"                             ! Prefix 0
"Orb"                               ! Prefix 1
"World"                             ! Prefix 2
**********************************  IPC 2   ****************************
TX                                  ! IPC Mode (OFF,TX,RX,TXRX,ACS,WRITEFILE,READFILE)
0                                   ! AC.ID for ACS mode
"State00.42"                        ! File name for WRITE or READ
SERVER                              ! Socket Role (SERVER,CLIENT,GMSEC_CLIENT)
192.168.100.12     10001            ! Server Host Name, Port
TRUE                                ! Allow Blocking (i.e. wait on RX)
TRUE                                ! Echo to stdout
3                                   ! Number of TX prefixes
"SC[1]"                             ! Prefix 0
"Orb"                               ! Prefix 1
"World"                             ! Prefix 2
```

Example for RX (Inp_IPC.txt)

```
<<<<<<<<<<<<<< 42: InterProcess Comm Configuration File >>>>>>>>>>>>>>
2                                   ! Number of Sockets
**********************************  IPC 1   ****************************
RX                                  ! IPC Mode (OFF,TX,RX,TXRX,ACS,WRITEFILE,READFI
0                                   ! AC.ID for ACS mode
"State00.42"                        ! File name for WRITE or READ
CLIENT                              ! Socket Role (SERVER,CLIENT,GMSEC_CLIENT)
192.168.100.42   10002              ! Server Host Name, Port
TRUE                                ! Allow Blocking (i.e. wait on RX)
TRUE                                ! Echo to stdout
1                                   ! Number of TX prefixes
"SC[0]"                             ! PREFIX sat1
**********************************  IPC 2   ****************************
RX                                  ! IPC Mode (OFF,TX,RX,TXRX,ACS,WRITEFILE,READFI
0                                   ! AC.ID for ACS mode
"State00.42"                        ! File name for WRITE or READ
CLIENT                              ! Socket Role (SERVER,CLIENT,GMSEC_CLIENT)
192.168.100.42   10003              ! Server Host Name, Port
TRUE                                ! Allow Blocking (i.e. wait on RX)
TRUE                                ! Echo to stdout
1                                   ! Number of TX prefixes
"SC[1]"                             ! PREFIX sat2
```

6. Modify Inp_Cmp.txt

```
<<<<<<<<<<<<<<<  42:  Command Script File  >>>>>>>>>>>>>>>
0.0 SC[0] Cmd Angles = [0.0 0.0 0.0] deg, Seq = 123 wrt L Frame
0.0 SC[1] Cmd Angles = [0.0 0.0 0.0] deg, Seq = 123 wrt L Frame
EOF

##################################################################
```

7. To show the reference frame see Inp_Graphics.txt (for 42 Cam and 42 Map)

```
FALSE                               !  Pause at Startup
TRACK_HOST                          !  POV Mode (TRACK_HOST, TRACK_TARGET, FIXED_IN
SC                                  !  Host Type (WORLD, REFORB, FRM, SC, BODY)
0  0  L                             !  Initial Host SC, Body, POV Frame
SC                                  !  Target Type (WORLD, REFORB, FRM, SC, BODY)
0  0  N                             !  Initial Target SC, Body, POV Frame
NEG_Y                               !  Boresight Axis
POS_Z                               !  Up Axis
```

8. Modify launch.sh to launch the new 42 configuration.

29.4 NOS3 Modified files:

Modified files script */Desktop/github-nos3/estrattore.sh* indicates most of the modified files in nos3 directories.

./components/generic_mag/fsw/src/generic_mag_app.h : GENERIC_MAG_PIPE_DEPTH
./components/generic_mag/fsw/platform_inc/generic_mag_platform_cfg.h : GENERIC_MAG_CFG_DELAY
./components/generic_star_tracker/fsw/src/generic_star_tracker_device.h : IsValid
./components/generic_star_tracker/fsw/src/generic_star_tracker_device.c :
GENERIC_STAR_TRACKER_ReadData
./components/generic_star_tracker/fsw/src/generic_star_tracker_msg.h : Generic_star_tracker
./components/generic_star_tracker/fsw/src/generic_star_tracker_msg.h : DeviceHK
./components/generic_star_tracker/fsw/src/generic_star_tracker_app.c : GENERIC_STAR_TRACKER_AppInit
./components/generic_star_tracker/fsw/src/generic_star_tracker_app.c :
GENERIC_STAR_TRACKER_ReportHousekeeping
./components/generic_star_tracker/fsw/src/generic_star_tracker_app.c :
GENERIC_STAR_TRACKER_ReportDeviceTelemetry
./components/generic_star_tracker/fsw/src/generic_star_tracker_app.c : GENERIC_STAR_TRACKER_Enable
./components/generic_star_tracker/fsw/src/generic_star_tracker_app.c : GENERIC_STAR_TRACKER_Disable
./components/generic_star_tracker/fsw/src/generic_star_tracker_app.h : DevicePkt
./components/generic_star_tracker/fsw/platform_inc/generic_star_tracker_platform_cfg.h :
_GENERIC_STAR_TRACKER_PLATFORM_CFG_H_
./components/isl/fsw/src/isl_app.c :
./components/isl/fsw/src/isl_app.c : ISL_AppData
./components/isl/fsw/src/isl_app.c : ISL_AppInit
./components/isl/fsw/src/isl_app.c : frameBuffer
./components/isl/fsw/src/isl_device.c : ISL_ReadData
./components/isl/fsw/src/isl_device.h : DropPktCounter
./components/isl/fsw/src/isl_device.h : DeviceCounter
./components/cryptolib/support/standalone/standalone.c : TMTF_UpdateErrCtrlField
./components/cryptolib/support/standalone/standalone.c : crypto_standalone_tc_apply
./components/cryptolib/support/standalone/standalone.c : crypto_standalone_tm_process
./components/cryptolib/src/sa/internal/sa_interface_inmemory.template.c : sa_config
./components/cryptolib/src/sa/internal/sa_interface_inmemory.template.c : sa_start
./components/cryptolib/src/sa/internal/sa_interface_inmemory.template.c : sa_rekey
./components/cryptolib/src/sa/internal/sa_interface_inmemory.template.c : sa_create
./components/cryptolib/src/core/crypto_config.c : Crypto_Init_TC_Unit_Test
./components/cryptolib/src/core/crypto_key_mgmt.c : Crypto_Key_OTAR
./components/cryptolib/src/core/crypto_tc.c : Crypto_TC_ApplySecurity_Cam
./components/cryptolib/src/core/crypto_tc.c : Crypto_TC_ProcessSecurity_Cam
./components/cryptolib/src/core/crypto_tc.c : Crypto_Prepare_TC_AAD
./components/cryptolib/src/core/crypto.c : Crypto_Get_Managed_Parameters_For_Gvcid
./components/cryptolib/src/core/crypto.c : Crypto_Process_Extended_Procedure_Pdu
./components/cryptolib/src/crypto/libgcrypt/cryptography_interface_libgcrypt.template.c :
./components/cryptolib/src/crypto/libgcrypt/cryptography_interface_libgcrypt.template.c :
cryptography_aead_decrypt
./components/cryptolib/include/crypto.h : TC_BLOCK_SIZE
./components/cryptolib/include/crypto_config.h : KEY_SIZE
./components/cryptolib/include/crypto_config.h : KEY_ID_SIZE
./components/cryptolib/include/crypto_config.h : IV_SIZE
./components/cryptolib/include/crypto_config.h : IV_SIZE_TC
./components/arducam/fsw/src/cam_lib.h : CAM_TIMEOUT
./components/arducam/fsw/src/cam_lib.c : arducam_i2c_write_regs
./components/arducam/fsw/src/cam_app.h : CAM_PIPE_DEPTH
./components/arducam/fsw/src/cam_child.c : CAM_fifo
./components/novatel_oem615/fsw/src/novatel_oem615_device.c :
NOVATEL_OEM615_CommandDeviceCustom
./components/novatel_oem615/fsw/src/novatel_oem615_device.c : NOVATEL_OEM615_ReadHK
./components/novatel_oem615/fsw/src/novatel_oem615_device.c : NOVATEL_OEM615_RequestData

./components/novatel_oem615/fsw/src/novatel_oem615_device.c : NOVATEL_OEM615_ChildProcessReadData
./components/novatel_oem615/fsw/src/novatel_oem615_app.c : NOVATEL_OEM615_ChildProcessRequestData
./components/novatel_oem615/fsw/src/novatel_oem615_child.h : NOVATEL_OEM615_READ_TIMEOUT
./components/novatel_oem615/fsw/platform_inc/novatel_oem615_platform_cfg.h :
NOVATEL_OEM615_CFG_MS_TIMEOUT
./components/generic_adcs/fsw/src/generic_adcs_msg.h : MAC
./components/generic_adcs/fsw/src/generic_adcs_msg.h : Generic_ADCS_DI_Gps_Tlm_Payload_t
./components/generic_adcs/fsw/src/generic_adcs_msg.h : Generic_ADCS_DI_Star_Tracker_Tlm_Payload_t
./components/generic_adcs/fsw/src/generic_adcs_msg.h : Gps
./components/generic_adcs/fsw/src/generic_adcs_msg.h : Star_Tracker
./components/generic_adcs/fsw/src/generic_adcs_msg.h : Generic_ADCS_AC_Normal_Tlm_t
./components/generic_adcs/fsw/src/generic_adcs_msg.h : Normal
./components/generic_adcs/fsw/src/generic_adcs_ingest.h : _GENERIC_ADCS_INGEST_H_
./components/generic_adcs/fsw/src/generic_adcs_ingest.c :
./components/generic_adcs/fsw/src/generic_adcs_ingest.c : Generic_ADCS_ingest_generic_Gps
./components/generic_adcs/fsw/src/generic_adcs_ingest.c : Generic_ADCS_ingest_generic_Star_Tracker
./components/generic_adcs/fsw/src/generic_adcs_adac.c :
./components/generic_adcs/fsw/src/generic_adcs_adac.c :
Generic_ADCS_execute_attitude_determination_and_attitude_control
./components/generic_adcs/fsw/src/generic_adcs_adac.c : AD_imu
./components/generic_adcs/fsw/src/generic_adcs_adac.c : AD_sol
./components/generic_adcs/fsw/src/generic_adcs_adac.c : AD_to_GNC
./components/generic_adcs/fsw/src/generic_adcs_adac.c : AC_normal
./components/generic_adcs/fsw/src/generic_adcs_app.c :
./components/generic_adcs/fsw/src/generic_adcs_app.c : Generic_ADCS_AppInit
./components/generic_adcs/fsw/src/generic_adcs_app.c : Generic_ADCS_ProcessCommandPacket
./components/generic_adcs/fsw/src/generic_adcs_app.c : Generic_ADCS_ProcessGroundCommand
./components/generic_fss/fsw/src/generic_fss_device.c : GENERIC_FSS_DEVICE_CMD_SIZE
./components/generic_fss/fsw/src/generic_fss_app.h : GENERIC_FSS_PIPE_DEPTH
./components/generic_fss/fsw/platform_inc/generic_fss_platform_cfg.h : GENERIC_FSS_CFG_DELAY
./fsw/apps/ci/fsw/examples/udp_tf/ci_custom.c :
./fsw/apps/ci/fsw/examples/udp_tf/ci_custom.c : g_CI_CustomData
./fsw/apps/ci/fsw/examples/udp_tf/ci_custom.c : CI_CustomReadCltuSocket
./fsw/apps/ci/fsw/examples/multi/ci_custom.c : CI_CustomMain
./fsw/apps/ci/fsw/examples/udp/ci_custom.c : CI_ForwardMsgToISL
./fsw/apps/io_lib/fsw/src/formats/tmtf.c : TMTF_UpdateErrCtrlField
./fsw/apps/io_lib/fsw/src/services/tm_sdlp.c : TM_SDLP_InitIdlePacket
./fsw/apps/io_lib/fsw/src/services/tm_sdlp.c : TM_SDLP_AddIdlePacket
./fsw/apps/io_lib/fsw/src/services/tm_sdlp.c : TM_SDLP_CompleteFrame
./fsw/apps/io_lib/fsw/src/services/tm_sdlp.c : TM_SDLP_AddData
./fsw/apps/cf/fsw/src/cf_cmd.h : CF_CMD_H
./fsw/apps/cf/fsw/src/cf_cmd.c : CF_ProcessGroundCommand
./fsw/apps/cf/fsw/src/cf_app.c : CF_Init
./fsw/apps/cf/fsw/src/cf_app.h : CF_EXTERNAL_CFDP
./fsw/apps/cf/fsw/src/cf_app.h : rcv_selector
./fsw/apps/cf/fsw/src/cf_app.h : SPP_Rcv
./fsw/apps/cf/fsw/src/cf_app.h : DeviceID
./fsw/apps/cf/fsw/inc/cf_msgids.h : CFDP_EXT_TLM_MID
./fsw/apps/to/fsw/src/to_app.h : TO_SCH_PIPE_DEPTH
./fsw/apps/to/fsw/src/to_app.h : TO_CMD_PIPE_DEPTH
./fsw/apps/to/fsw/src/to_app.h : TO_TLM_PIPE_DEPTH
./fsw/apps/to/fsw/src/to_app.c : TO_ProcessNewData
./fsw/apps/to/fsw/examples/udp_tf/to_platform_cfg.h : TO_SCH_PIPE_DEPTH
./fsw/apps/to/fsw/examples/udp_tf/to_platform_cfg.h : TO_CMD_PIPE_DEPTH
./fsw/apps/to/fsw/examples/udp_tf/to_platform_cfg.h : TO_TLM_PIPE_DEPTH
./fsw/apps/to/fsw/examples/udp_tf/to_platform_cfg.h : TO_WAKEUP_TIMEOUT
./fsw/apps/to/fsw/examples/udp_tf/to_platform_cfg.h : TO_CUSTOM_TF_SIZE
./fsw/apps/to/fsw/examples/udp_tf/to_platform_cfg.h : TO_CUSTOM_TF_ERR_CTRL
./fsw/apps/to/fsw/examples/multi_tf/to_platform_cfg.h : TO_TLM_PIPE_DEPTH

./fsw/apps/to/fsw/examples/rs422/to_platform_cfg.h : TO_TLM_PIPE_DEPTH
./fsw/apps/to/fsw/examples/multi/to_platform_cfg.h : TO_TLM_PIPE_DEPTH
./fsw/apps/to/fsw/examples/udp/to_platform_cfg.h : TO_TLM_PIPE_DEPTH
./fsw/psp/fsw/nos-linux/src/cfe_psp_start.c : TICKS_PER_SECOND
./fsw/cfe/modules/sb/fsw/src/cfe_sb_api.c :
./fsw/cfe/modules/sb/fsw/src/cfe_sb_api.c : CFE_SB_SubscribeFull
./fsw/cfe/modules/sb/fsw/src/cfe_sb_api.c : CFE_SB_TransmitMsg
./fsw/cfe/modules/sb/fsw/src/cfe_sb_api.c : CFE_SB_TransmitMsgValidate
./fsw/cfe/modules/sb/fsw/src/cfe_sb_api.c : CFE_SB_BroadcastBufferToRoute
./fsw/cfe/modules/sb/fsw/src/cfe_sb_init.c :
./fsw/cfe/modules/sb/fsw/src/cfe_sb_init.c : CFE_SB_MemPoolDefSize
./fsw/cfe/modules/sb/fsw/src/cfe_sb_init.c : CFE_SB_EarlyInit
./fsw/cfe/modules/es/fsw/src/cfe_es_task.c : CFE_ES_TaskPipe
./fsw/cfe/modules/es/fsw/src/cfe_es_start.c :
./fsw/cfe/modules/es/fsw/src/cfe_es_start.c : CFE_ES_PANIC_DELAY
./fsw/cfe/modules/es/fsw/src/cfe_es_start.c : CFE_ES_Main
./fsw/cfe/modules/es/fsw/inc/cfe_es_msg.h : CFE_ES_StopAppCmd
./fsw/cfe/modules/es/fsw/inc/cfe_es_msg.h : CFE_ES_StopAppCmd_t
./fsw/cfe/modules/evs/fsw/src/cfe_evs_utils.c : EVS_IsFiltered
./fsw/cfe/cmake/sample_defs/sample_mission_cfg.h : CFE_MISSION_MAX_API_LEN
./fsw/build/exe/host/crypto.h : TC_BLOCK_SIZE
./fsw/build/exe/host/crypto_config.h : KEY_SIZE
./fsw/build/exe/host/crypto_config.h : KEY_ID_SIZE
./fsw/build/exe/host/crypto_config.h : IV_SIZE
./fsw/build/exe/host/crypto_config.h : IV_SIZE_TC
./fsw/nos3_defs/tables/to_config.c :
./fsw/nos3_defs/tables/sch_def_msgtbl.c :
./fsw/nos3_defs/tables/sch_def_schtbl.c :
./fsw/nos3_defs/tables/sch_def_schtbl.c : SCH_DefaultScheduleTable
./fsw/nos3_defs/nos3_mission_cfg.h : CFE_MISSION_MAX_API_LEN
./fsw/nos3_defs/cpu1_platform_cfg.h : CFE_PLATFORM_SB_DEFAULT_MSG_LIMIT
./fsw/nos3_defs/cpu1_platform_cfg.h : CFE_PLATFORM_ES_STARTUP_SCRIPT_TIMEOUT_MSEC